

Striver

Dsa

Sheet

"180"

Part - 2

Implement Stack Using Arrays : Problem-74

push (6)

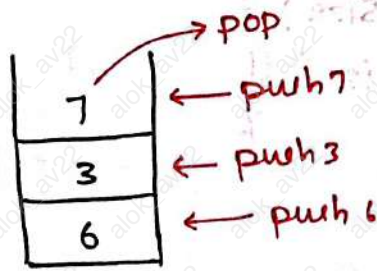
push (3)

push (7)

top() → 7

pop → POP 7

Size - 2



Size = 1000;

```
int arr[] = new int [size];
```

```
int top = -1; // initially
```

```
void push (int x) {
```

```
    top ++;
```

```
    arr[top] = x;
```

```
}
```

```
int pop () {
```

```
    int x = arr[top];
```

```
    top --;
```

```
    return x;
```

```
}
```

```
int top () {
```

```
    return arr[top];
```

```
}
```

```
int size () {
```

```
    return top + 1;
```

```
}
```

Time complexity - $O(N)$

Space complexity - $O(N)$

Implement Queue using arrays : Problem-75

```
private int arr[];
```

```
private int start, end, curSize, maxSize;
```

```
public Queue () {
```

```
    arr = new int [16];
```

```
    start = -1;
```

```
    end = -1;
```

```
    curSize = 0;
```

```
}
```

```
public Queue (int maxSize) {
```

```
    this.maxSize = maxSize;
```

```
    arr = new int [maxSize];
```

```
    start = -1;
```

```
    end = -1;
```

```
    curSize = 0;
```

```
}
```

```
public void push(int newElement) {
```

```
    if (currSize == maxSize) {  
        mean Queue is full;  
    }
```

```
    if (end == -1) {  
        start = 0;  
        end = 0;  
    }
```

```
    else  
        end = (end + 1) % maxSize;
```

```
    arr[end] = newElement;
```

```
    currSize ++;
```

```
}
```

```
public int pop() {
```

```
    if (start == -1) {  
        mean Queue Empty  
    }
```

```
    int popeed = arr[start];
```

```
    if (currSize == 1)  
        start = -1;  
        end = -1;
```

```
    else  
        start = (start + 1) % maxSize;
```

```
    currSize --;
```

```
    return popeed;
```

```
}
```

```
public int top() {
```

```
    if (start == -1)  
        mean queue Empty
```

```
    return arr[start];
```

```
}
```

```
public int size() {
```

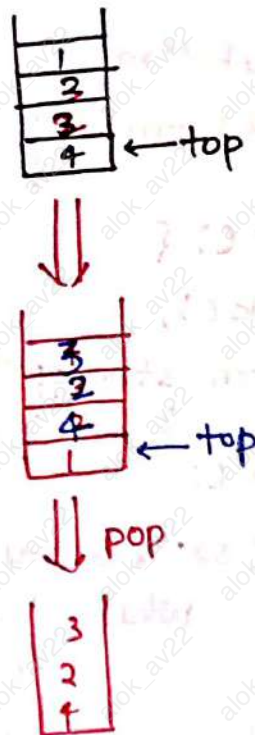
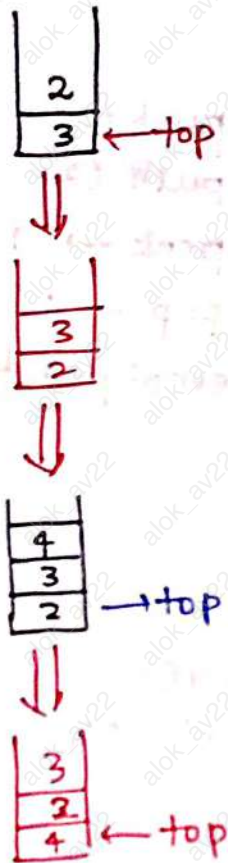
```
    return currSize;
```

```
}
```

Tc - $O(N)$

Implement stack using Queue. : Problem - 76

push (3)
push (2)
push (4)
push (1)
top - ①
pop - (1)
top () → ④



after pushing a new element remove all the element from the queue and again put into the queue.

```
Queue <Integer> q;
```

```
void push (int x) {
```

```
    q.add(x);
```

```
    for (i=0; i < q.size()-1; i++) {
```

```
        q.add(q.remove());
```

```
    }
```

```
}
```

```
int pop ()
```

```
    return q.remove();
```

```
int top ()
```

```
    return q.peek();
```

```
int size ()
```

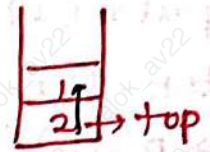
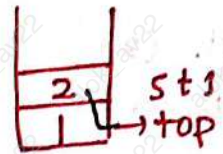
```
    return q.size();
```

Tc - $O(N)$

Sc - $O(N)$

Implement Queue Using Stacks : Problem : 7.7

```
Stack<int> s1();  
Stack<int> s2();  
public void push (int x) {  
    s1.push(x);  
}  
  
public int pop() {  
    peek();  
    return s2.pop();  
}  
  
public int peek() {  
    if (s2.isEmpty())  
        while (!s1.isEmpty())  
            s2.push(s1.pop());  
    return s2.peek();  
}  
  
public boolean empty() {  
    return s2.isEmpty() && s1.isEmpty();  
}
```



SC - $O(2N)$
TC - $O(N)$

Check for Balance parentheses : Problem : 7.8

First put all the open brackets into stack and if any closing brackets arrives match with the top of stack and remove. In the last check the size of stack == 0 & it is balance otherwise no.

SC - $O(N)$
TC - $O(N)$

Next Greater Element : Problem-79

$N=4$, $arr[] = [1, 3, 2, 4]$

output $[3, 4, 4, -1]$

traverse the array from last to 0th index.

```
Stack<int> st();
long nge[] = new int[n];
for (i = arr.length - 1; i >= 0; i--) {
    if (!st.isEmpty()) {
        while (!st.isEmpty() && st.peek() <= arr[i]) {
            st.pop();
        }
    }
    if (st.isEmpty()) {
        nge[i] = -1;
    }
    else {
        nge[i] = st.peek();
    }
    st.push(arr[i]);
}
```

SC - $O(N)$

TC - $O(N)$

this is the optimal Approach.

In Brute just select a number and then find the greater number than the selected number if you get then push in the array. otherwise push -1.

Time complexity - $O(N^2)$.

$[1, 3, 2, 4]$

nge

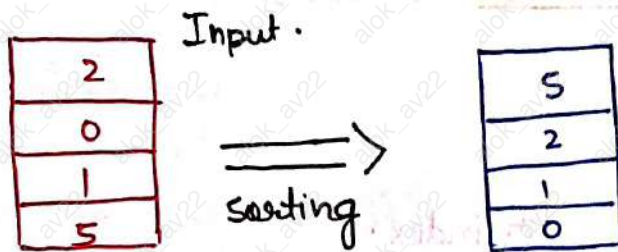
3	4	4	-1
---	---	---	----

3
2
4

initially stack size = 0

$3 > 2$ (false) remove 2

Sort a Stack - % Problem - 80



print - 5, 2, 1, 0

main (stack <int> s) { $\rightarrow$ fun name - sortStack

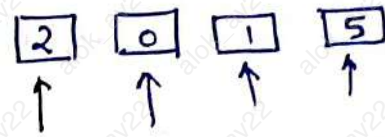
if (!s.isEmpty()) {

int x = s.pop();

sortStack (s);

sortedInsert (s, x);

}



sortedInsert (stack <int> s, int x) {

// if stack is Empty or st. peek is less than x)

// push that element.

if (st.isEmpty() || x > s.peek()) {

s.push (x);

return;

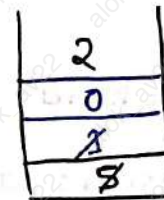
}

int temp = s.pop(x);

sortedInsert (s, x);

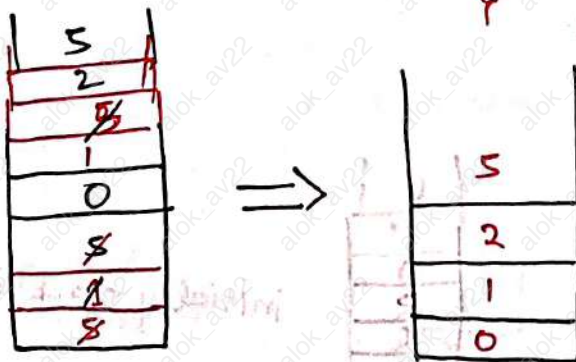
s.push (temp);

}



temp = 5

temp = 1



temp = 5
temp = 1

temp = 5
temp = 1

Hypothesis

- ① Pop element
- ② call the function (himself)
- ③ Sorted insert fun (s, x)

LRU cache - Problem - 82

- LRU cache (int capacity) - we need to initialize the LRU cache with positive size capacity.
- int get (int key) - return the val of key if the key exists otherwise return -1;
- void put (int key, int value) - Update the val. of key if key exists. Otherwise, add the key-val pair to cache. if no of keys exceeds the capacity from this operation, evict the least recent.

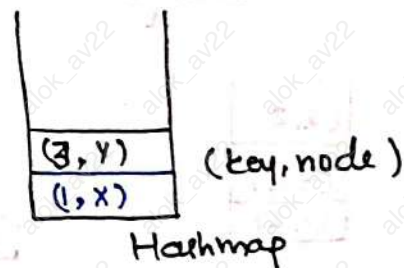
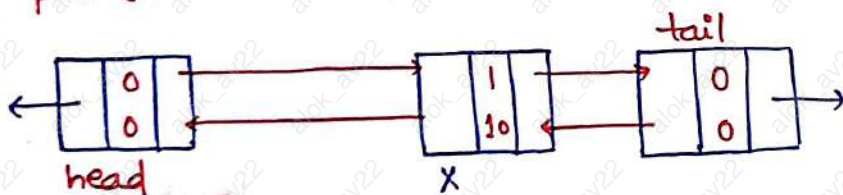
Intuition - While inserting {key, val} pair into DDL make sure that we are inserting it from the back tail to head.

Size = 3 put(1, 10) put(3, 15) put(2, 12) get(3) put(4, 25)

Create a DLL and map.



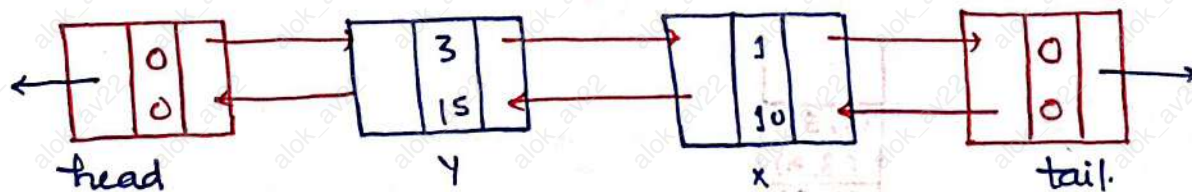
put (1, 10) Not present so put



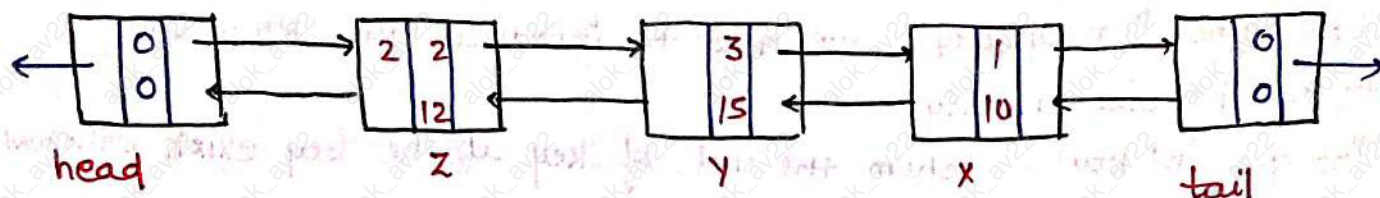
While inserting consider:

- ① Check if {key, val} is already present
- ② Check for capacity if capacity == size so remove the pair which is just right after head node. and add
- ③ if key is not present return -1

put (3, 15) Not present so put



put (2,12) Not present so put.

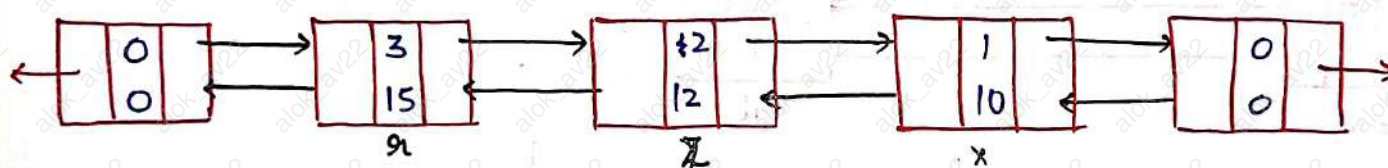
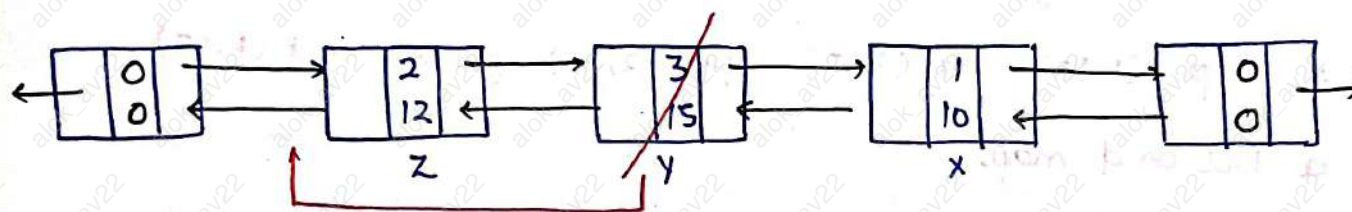


(2, z)
(3, y)
(1, x)

get(3), yes, so it is present.

Now most recent element should be change.

Delete the Node and add it right after head node.



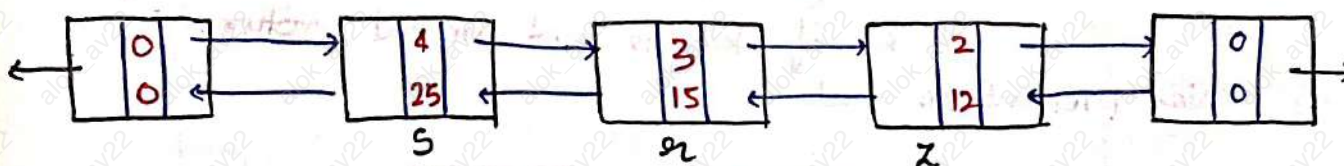
(2, z)
(3, y)
(1, x)

(3, y)
(2, z)
(1, x)

put (4,25) because capacity full. so we have to remove recent used element.

1
10

will remove. and add (4,25)



(4, s)
(3, y)
(2, z)


```
class LRUCache {
```

```
    Node head = new Node (0,0);
```

```
    Node tail = new Node (0,0);
```

```
    Map < int, Node > map = new HashMap ();
```

```
    int capacity;
```

```
    public LRUCache (int - capacity) {
```

```
        capacity = - capacity;
```

```
        head.next = tail;
```

```
        tail.prev = head;
```

```
    }
```

```
class Node {
```

```
    Node prev, next;
```

```
    int key, val;
```

```
    Node (int - key, int - value) {
```

```
        key = - key;
```

```
        value = - value;
```

```
    }
```

```
}
```

```
public int get (int key) {
```

```
    if (map.containsKey (key)) {
```

```
        Node node = map.get (key);
```

```
        remove (node);
```

```
        insert (node);
```

```
        return node.value;
```

```
    }
```

```
    else return -1;
```

```
}
```

```
public void put (int key, int val) {
```

```
    if (map.containsKey (key)) {
```

```
        remove (map.get (key));
```

```
    }
```

```
    if (map.size == capacity) {
```

```
        remove (map.tail.prev);
```

```
    }
```

```
    insert (new Node (key, val));
```

```
}
```

```
private void remove (Node node) {
    map.remove (node.key);
    node.prev.next = node.next;
    node.next.prev = node.prev;
}
```

```
private void insert (Node node) {
    map.put (node.key, value) node);
    node.next = head.next;
    node.next.prev = node;
    head.next = node;
    node.prev = head;
}
```

Time complexity - $O(N)$

Space complexity - $O(1)$

LFU Cache (Least frequent used) : Problem 83

get(key) → get the value of key if exists, else - 1

put (key, val) → update the value of key if it is present, or insert the key if not present.

When the cache is full, it removes the LFU one if there are multiple LFU so we choose with the help of LRU.

Size = 2

put (1, 10)

put (2, 20)

get (1) → 10

put (3, 30)

get (2) → -1

get (3) → 30

put (4, 4)

get (1) → -1

get (3) → 30

get (4) → 4

freq.

① → (1, 10) (2, 20)

2 → (1, 10)

Now cache is full and remove (2, 20)

1 → (3, 30)

2 → (1, 10) (3, 30)

because Here is a tie so we remove one of them by taking help of LRU.

1 → (4, 4)

2 → (4, 4) (3, 30) (4, 4)

3 → (3, 30)

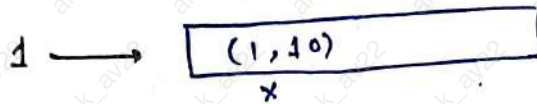
→ DLL
 Map < freq, list > freqlist;
 Map < key, node > keynode;
 capacity = 0
 freq = 0

3
(3, 2)
(2, 4)
(1, x)

(key, node)

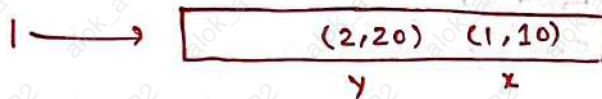
Size = 3

put (1, 10), check that (1, 10) is present in the keynode map or not
 if it is not so check for the capacity.



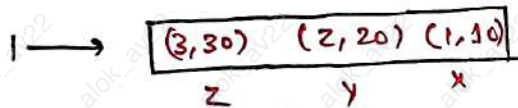
freq = 1
 capacity = 1

Now put (2, 20) → verify it is present in the map or not also check the capacity.



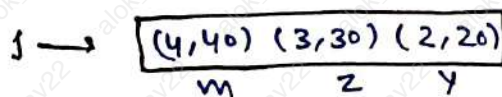
freq = 1
 capacity = 2

put (3, 30) → verify is present in the map or not.



freq = 1
 capacity = 3.

put (4, 40) → it is not present in the map but capacity is full
 so we have to remove, so we remove LFU element. but
 because it has three elements so choose according to LRU.



(3, 2)
(2, 4)
(1, x)

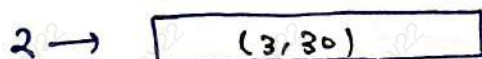
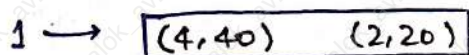
→

(4, m)
(3, 2)
(2, 4)

get(3) → yes it is in the map.

and check for the value of 3. it is 30.

and now we have to delete the z node. frequency will be 2.



get(2) → yes it is exist.
the value return. 20

and after that we have to remove the value (2, 20)
and put it into freq. 2.

1 → (4, 40)

2 → (2, 20) (3, 30)

get(4) → it is exist. val = 40

1 →

2 → (4, 40) (2, 20) (3, 30)

freq. 1 has a empty list
so we have to increase the
freq.

freq = 2
Size = 3

put(5, 50) → No we can insert
it, but capacity is full.

1 → (5, 50)

2 → (4, 40) (2, 20)

(4, W)
(5, X)
2, Y

freq = 1

put(2, 25) → 2 is exist so we update it and change
freq.

1 → (5, 50)

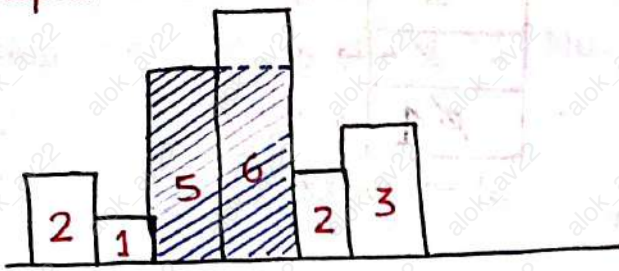
2 → (4, 40)

3 → (2, 25)

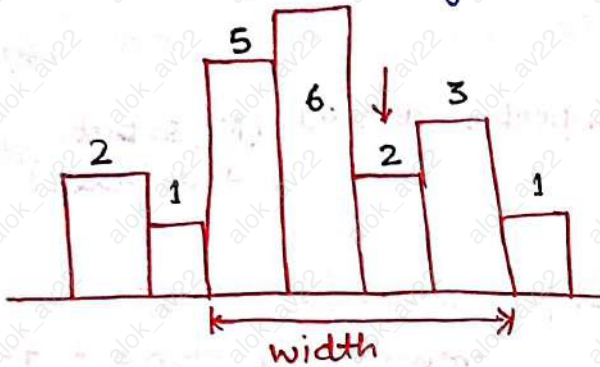
Largest Rectangle in a histogram - Problem - 04

Input $\Rightarrow$ heights = [2, 1, 5, 6, 2, 3]

Output = 10



Total rectangle len = 10, width of each height = 1



Brute force approach:

Find the right smaller and left smaller element and find the largest rectangle area of the Histogram.

```
main(arr, n) {
```

```
    int maxArea = 0;
```

```
    for(i = 0; i < n; i++) {
```

```
        int minHeight = INT_MAX;
```

```
        for(j = i; j < n; j++) {
```

```
            minHeight = Math.min(minHeight, arr[j]);
```

```
            maxArea = Math.max(maxArea, minHeight * (j - i + 1));
```

```
        }
```

```
    }
```

```
}
```

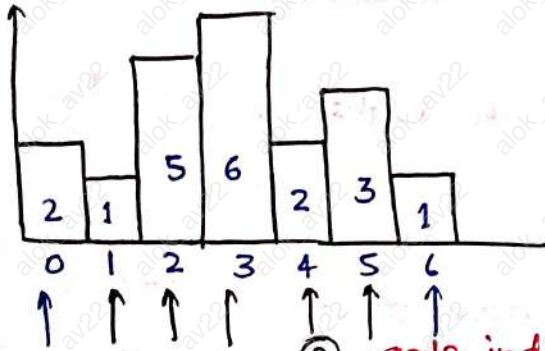
Time complexity - $O(N^2)$

Optimised Approach. The intuition behind the approach is the same as finding the smaller element on the both side but in an optimised way using the concept of next greater element & next smaller element.

0	0	2	3	2	5	0
0	1	2	3	4	5	6

Left small

8	→	3
4	→	2
3	→	6
2	→	5
0	→	1
st → index		



① check $st.peek > arr[0]$ (No, so put the index)
 ② goto index - 1 and

③ goto idx - 2 and
 Now $st.peek > arr[2]$ (No)
 and put the idx into stack.

$st.peek > arr[1]$ (Yes, so remove the element and put the current idx.)

④ goto idx → 4

$st.peek > arr[4]$ (Yes) remove 3, 2 from stack because they both are greater

right smaller -

0	6	3	3	5	5	6
0	1	2	3	4	5	6

① on idx - 6 st is empty put the curr idx in the array. and put 6 inst.

② on idx - 5

$st.peek > arr[5]$, No
 put the idx into stack and
 store the idx in the array.

1	→	1
2	→	5
3	→	6
4	→	2
5	→	3
6	→	1
st → idx		

③ No when we go to idx 4 val 2.

$arr[st.peek] > arr[4]$ Yes, so we have to remove the value from st. and put the idx in arr

and stack.

Now 6 is the boundary so put $6-1=5$ in the arr.
because 6th idx is smaller.

Now $st.peek > arr[3]$ No so put it into st and arr

Now
go to 2

$st.peek > arr[2]$, yes so remove the 3 from
stack.

Now after this for idx-1 st will empty so put the last idx.

for 0th index -

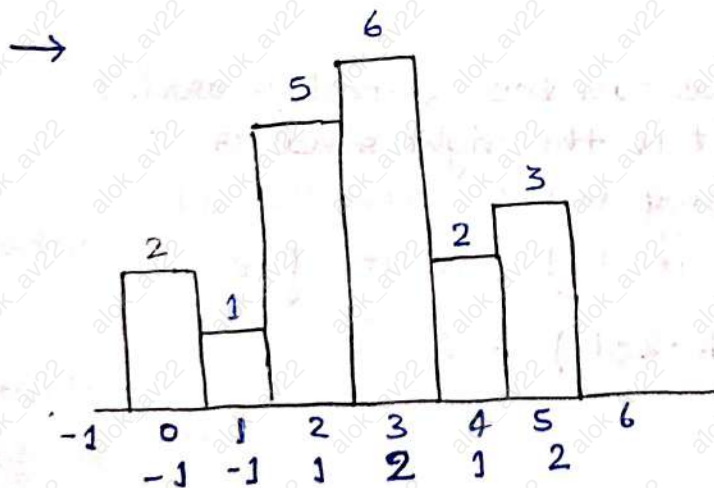
$$(\text{right smaller} - \text{left smaller}^{+1}) * a[i]_{idx}$$

$$Tc - O(N) + O(N) \approx O(N)$$

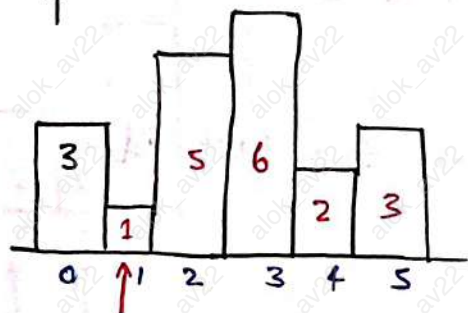
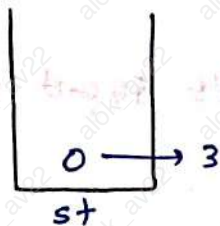
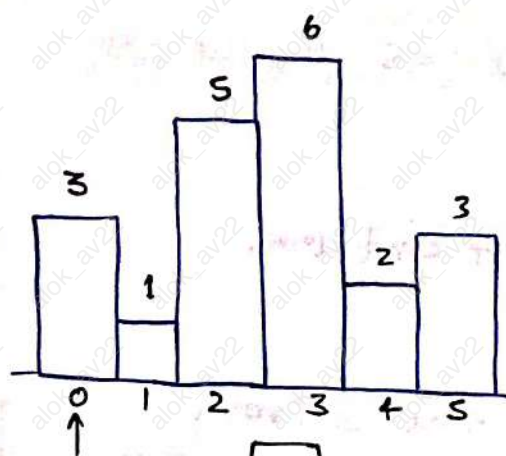
$$Sc - O(3N)$$

Optimised solution - 2

— pop $\rightarrow$ bigger or same value
 $\rightarrow$ ans $\rightarrow$ push self idx.



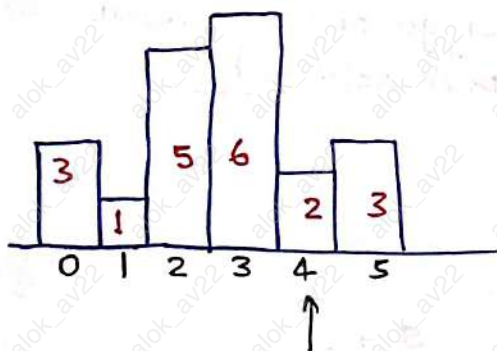
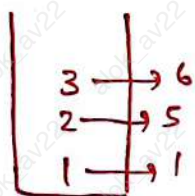
5	$\rightarrow$ 3
4	$\rightarrow$ 2
3	$\rightarrow$ 6
2	$\rightarrow$ 5
1	$\rightarrow$ 1
0	$\rightarrow$ 6
-1	$\rightarrow$ 2



current val is greater than $st \cdot peek$
so pop it from st and put the curr idx.

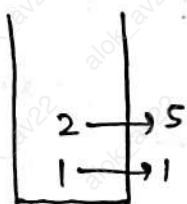


idx 2 and 3 val is greater than $arr[st \cdot peek]$ so simply put it into st .



Now we can see $st \cdot peek > arr[i]$
mean it is the right smaller.
so popping out 6 from the st .
and 5 is left smaller for 6.

$$MaxArea = \max(MaxArea, (6 * (4 - 2 - 1))) = 6$$

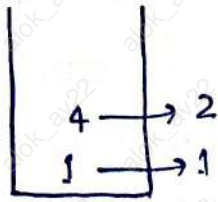


again $st \cdot peek > arr[i]$

Since 2 is smaller than 5 so it means it means it is right smaller

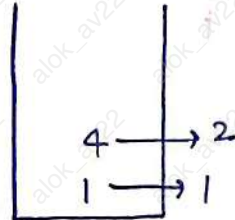
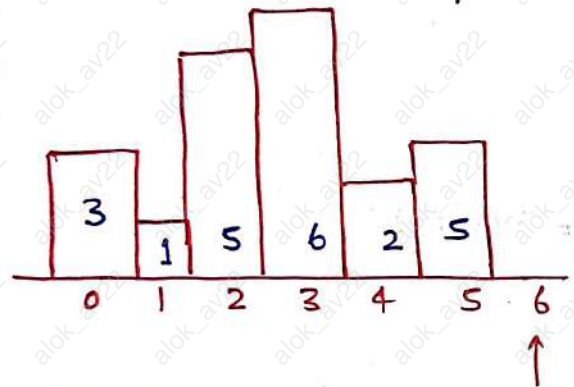
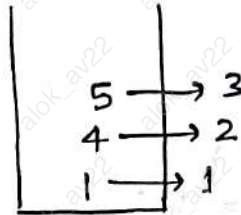
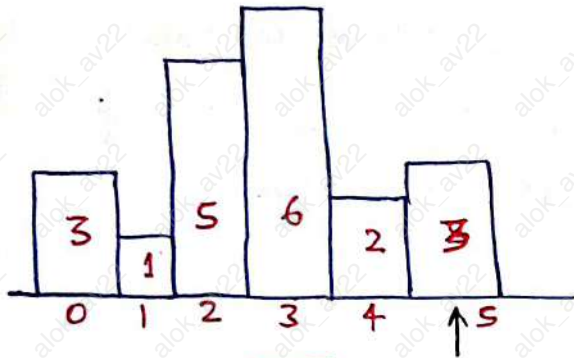
and popping out 5 from the stack, 1 is left smaller for 5

$$\text{MaxArea} = \max(6, (5 * (4 - 1 - 1))) = 10$$



Now 1 is smaller than 2 so we directly

push 2

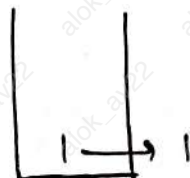


Now we will do one more iteration and consider 6 as a right smaller and so pop out 3 and consider 2 as left smaller.

$$\text{MaxArea} = (10, (3 * (6 - 4 - 1))) = 40$$

consider 6 is right smaller for 4 so pop 4 and 1 is left smaller.

$$\text{MaxArea} = 2 * (6 - 1 - 1) = 8, \quad \underline{\text{MaxArea} = 10}$$



Now pop out 1 6 is the right smaller and 0 is the left smaller

$$\text{MaxArea} = (10, 1 * (6 - 0 - 1)) = 10$$

```
main (arr) {
```

```
Stack <int> st;
```

```
int MaxA = 0;
```

```
for (i = 0; i < n; i++) {
```

```
    while (!st.isEmpty() && (i == n || arr[st.peek()] >= arr[i])) {
```

```
        int height = arr[st.peek()];
```

```
        st.pop();
```

```
        int width;
```

```
        if (st.isEmpty()) {
```

```
            width = i;
```

```
        }
```

```
        else {
```

```
            width = i - st.peek() - 1;
```

```
        }
```

```
        MaxA = Math.max(MaxA, width * height);
```

```
    }
```

```
        st.push(i);
```

```
    }
```

```
return MaxA;
```

```
}
```

Time complexity - $O(N) + O(N)$

Space - $O(N)$

Sliding window Maximum : Problem - 05

We use the intuition of NGE here.

```
main(arr) {
```

```
    Stack <int> st();
```

```
    int [] nge [] = new int [arr.length];
```

```
    st.push(arr.length - 1);
```

```
    nge[arr.length - 1] = arr.length;
```

```
    for (i = arr.length - 2; i >= 0; i--) {
```

```
        while (st.size() > 0 && arr[i] >= arr[st.peek()]) {
```

```
            st.pop();
```

```
        if (st.isEmpty()) {
```

```
            nge[i] = arr.length;
```

```
        else {
```

```
            nge[i] = st.peek();
```

```
        }
```

```
        st.push(i);
```

```
    }
```

```
    for (i = 0; i < arr.length - k; i++) {
```

```
        int j = i;
```

```
        while (nge[j] < i + k) {
```

```
            j = nge[j];
```

```
        }
```

```
        print(arr[j]);
```

```
    }
```

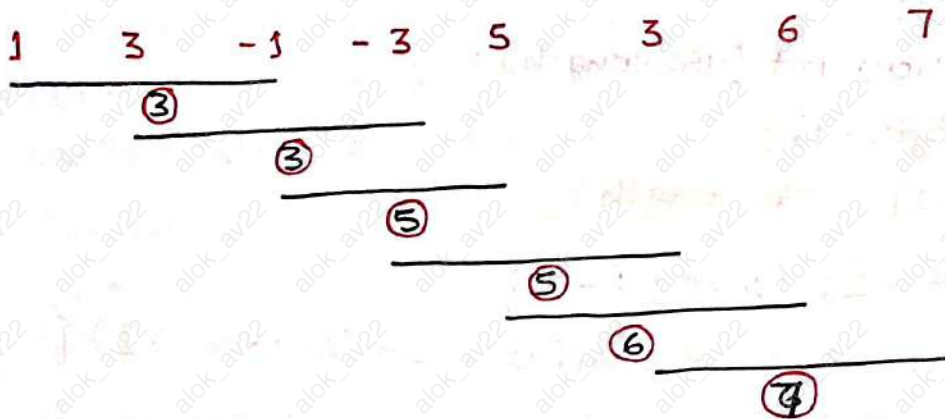
Time complexity - $O(N) + O(N)$

Space complexity - $O(2N)$.

Approach-2

nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3

Output = [3, 3, 5, 5, 6, 7]



Naive solution - traverse from 0 to give $(n-k)$ so we can get the max in the every window.

Now firstly traverse from 0 to 2 and find the max from it and put into the array.

```
for (i = 0 → 5) {  
    maxi = arr[i];  
    for (j = i + k - 1) {  
        maxi = max(arr[j], maxi);  
    }  
}
```

Time complexity - $O(N^2)$

Space - $O(1)$

Optimised Approach

We use the concept of NGE but

over here we store the elements in decreasing order. ↓
so we need a deque.

It allows



	1	3	-1	-3	5	3	6	7
idx	0	1	2	3	4	5	6	7
	↑	↑	↑	↑	↑			

In the 0th idx check does the deque has any thing → No. So push back the val 0

0	
---	--

then go to idx 1.

So now compare that peek element of dq. which is 1 is less than 3.

1 < 3 which can be my answer.

take this 0 out after this dq. be empty so push 1 into dq.

1	
---	--

Move to the 2nd idx. with val -1

Now dq. peek > arr[i] so no need to pop because we are maintaining the decreasing order

1	-1
---	----

Now we can see 2 is the last index of the subarray of size k.

So, if i look from the front the max element will be on the front.

Now pointer will move on to 3rd idx and. if we check $arr[dq.peek] > arr[i]$ so we can push into dq maintaining the decreasing order.

3	
---	--

ans

1	2	3
---	---	---

of size 3.

Now we have one more subarray

3	3
---	---

ans

Now move to the 4th idx.

0	1	2	3	4	5	6	7
1	3	-1	-3	5	3	6	7

Now my boundary is 2 to 4 but dq contains 3 so we have to pop 1.

1	2	3	
--------------	---	---	--

3	3	
---	---	--

ans.

Now $arr[dq.peek] < arr[i]$ so we have to pop 3 from dq.

1	2	3	4
--------------	--------------	--------------	---

this 3 to 5 my boundary so check is there any idx in the dq out of the boundary $\rightarrow$ No

3	3	5	5		
---	---	---	---	--	--

ans.

also $arr[dq.peek] > arr[i]$ so push

4	5	
---	---	--

Now boundary 4-6 No element in the dq is out of boundary. but $dq.peek < arr[i]$ pop both elements.

4	5	6	
--------------	--------------	---	--

3	3	5	5	6	7
---	---	---	---	---	---

Move to 7 and boundary 5-7 everything in the dq. is in the boundary.

but $dq.peek < arr[i] \rightarrow$ pop 6.

6	7	
--------------	---	--

check from front so it 7 and correspond to 7 so push it into the array.

--	--	--	--

Time complexity - $O(N) + O(N) \approx O(N)$

Sc - $O(K)$

```
main (arr, k) {
```

```
    int xi = 0;
```

```
    int ans[] = new int [n-k+1];
```

```
    Deque<int> dq;
```

```
    for (i = 0; i < arr.length; i++) {
```

```
        if (!dq.isEmpty() && dq.peek() == i - k) {
```

```
            dq.poll();
```

```
        }
```

// remove numbers
out of range k.

// remove smaller numbers in k range as they are useless.

```
        while (!dq.isEmpty() && arr[dq.peekLast()] < arr[i])
```

```
            dq.pollLast();
```

```
        dq.offer(i);
```

```
        if (i >= k - 1) {
```

```
            ans[xi++] = arr[dq.peek()];
```

```
        }
```

```
    }
```

```
    return ans;
```

// any i which is greater than k-1
be the max in the window because
it will be max because we are
storing everything in decreasing
order.

Implement Min Stack :-

push(-2)	getMin()
push(0)	pop()
push(-3)	getMin()

(-2, -2)

Brute force - The Naive approach is we have to push the pair and val, min element till now, on the stack.

(-2, -2), (0, -2), (-3, -3)

we push all the elements.

Now getMin() = -3

if pop so st.

(-2, -2), (0, -2)

getMin() = -2

Tc = $O(1)$

Sc = $O(2N)$

Optimal solution -

push(5)
push(3)

Initially min Element will be max set

first we push 5 into the stack because it is empty.

5

min element = 5

Now we have to push 3 so there can arrive two conditions.

$3 \geq \text{min element}$

||

$3 < \text{min element}$

↓
current element

↓ if True

so directly push the current val into the stack

↓ if yes

so push $2 \times \text{val} - \text{min element}$ into stack and update the val.

$$3 < 5$$

so -

$$2 \times 3 - 5 = 1 \text{ so we have to push 1 into the}$$

stack.



it is works like a flag
(Anomaly)

$$\text{minelement} = \cancel{5} 3$$

Now if pop operation call so we have to check
 $\text{st.peek} < \text{minelement}$ so it means it was a flag
 that was pushed by us and it's mean the min
 element will change after pop operation

$$\text{st.peek} < \text{minelement}$$



$$\text{minelement} = 2 \times \text{minelement} - \text{st.peek}$$

Stack < long > st;

Long min;

public MinStack() {

min = Long.MAX - VALUE;

}

① void push (int val) {

Long x = Long.valueOf (val);

if (st.size == 0) st.push (x);

min = x;

else if (val >= min) {

st.push (x);

else {

st.push (2 * x - min);

min = x;

}

}

② void pop() {

if (st.size == 0) return;

Long val = st.pop();

if (val < min) {

min = 2 * min - val;

}

③ int top() {

Long val = st.peek();

if (val < min) {

return min.intValue();

}

return val.intValue();

}

Rotting Oranges :- Problem - 07

given $\rightarrow$ m x n Matrix
Size

0 $\rightarrow$ represent empty cell

1 $\rightarrow$ fresh orange

2 $\rightarrow$ Rotten orange

Every min any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten. Return the min no of min to rot all the orange. if impossible so -1.

2	1	1
0	1	0
0	0	1

min 0

Initially 2 is rotten orange

min 1

2	2	1
2	1	0
0	1	1

min 2

2	2	2
2	2	0
0	1	1

min 3

2	2	2
2	2	0
0	2	1

min 4

2	2	2
2	2	0
0	2	2

min Time = 4 minutes.

let's take a example to think the intuition -

	0	1	2
0	0	2	2
1	0	2	2
2	2	2	1

(1,1)
(2,2)
(1,2)
(0,3)
~~(2,1)~~
~~(0,2)~~
~~(2,0)~~
Q

cut = 2

$$x = 0 + 2 = 2$$

$$y = 1 + 0 = 1$$

$$x = 2 + 0 = 2 \rightarrow x$$

$$y = 0 - 1 = -1$$

$$\begin{aligned} x &= 2 + 1 = 3 \\ y &= 0 + 0 = 0 \end{aligned} \quad \left. \vphantom{\begin{aligned} x &= 2 + 1 = 3 \\ y &= 0 + 0 = 0 \end{aligned}} \right\} x$$

$$x = 2 + (-1) = 1$$

$$y = 0 + (0) = 0$$

became $grid[1][0] = 0$ & no change

$min = 1$

$$\begin{aligned} x &= 0 + 0 = 0 \\ y &= 2 + 1 = 3 \end{aligned} \quad \left. \vphantom{\begin{aligned} x &= 0 + 0 = 0 \\ y &= 2 + 1 = 3 \end{aligned}} \right\} x$$

$$x = 0 + 0 = 0$$

$$y = 2 - 1 = 1$$

push(0,1) into q.

$$x = 0 + 1 = 1$$

$$y = 2 + 0 = 2$$

push(1,2) into q.

$$x = 0 - 1 = -1$$

$$y = 2 + 0 = 2$$

$x \geq -1 < 0$

$$cnt = cnt + q.size$$

$$= 2 + 3 = 5$$

$min = 4$

$$x = 2 + 0 = 2$$

$$y = 1 + 1 = 2$$

push(2,2) into q.

$$x = 2 + 0 = 2$$

$$y = 1 - 1 = 0$$

x already vis.

$$x = 2 + (1) = 3$$

$$y = 1 + (0) = 1$$

x

$$x = 2 - 1 = 1$$

$$y = 1 + 0 = 1$$

x zero

pop $\rightarrow$ 0, 1

$$x = 0 + 0 = 0$$

$$y = 1 + 1 = 2$$

} already present

$$x = 0 + 0 = 0$$

$$y = 1 - 1 = 0$$

} Empty cell.

$$x = 0 + 1 = 1$$

$$y = 0 + 0 = 0$$

} put it (1, 1) into q.

Similarly, (1, 2) $\rightarrow$ change pop and all the elements will pop so it will take 2 min to rot all the oranges.

ans

```
main(grid[][]){
```

```
    if(grid == null || grid.length == 0) return 0;
```

```
    int n = grid.length;
```

```
    int m = grid[0].length;
```

```
    Queue<int[]> q = new LinkedList<>();
```

```
    int cnt-fresh = 0;
```

```
    for(i=0; i<n; i++){
```

```
        for(j=0; j<m; j++){
```

```
            if(grid[i][j] == 2){
```

```
                q.offer(new int[] {i, j});
```

```
            if(grid[i][j] == 0){
```

```
                cnt-fresh++;
```

```
            }
```

```
        }
```

```
    }
```


if (fresh_cnt == 0) return 0;

int cntMin = 0, cnt = 0;

int dx[] = {0, 0, 1, -1};

int dy[] = {1, -1, 0, 0};

while (!q.isEmpty()) {

int size = q.size();

cnt += size;

for (i = 0; i < size; i++) { int[] point = q.poll();

for (j = 0; j < 4; j++) {

int x = point[0] + dx[j];

int y = point[1] + dy[j];

if (x < 0 || y < 0 || x >= n || y >= m || grid[x][y] == 0 ||
grid[x][y] == 2) continue;

grid[x][y] = 2;

q.offer(new int[] {x, y});

if (q.size() != 0) cntMin++;

return cnt - fresh == cnt ? cntMin : -1;

Time complexity - $O(n \times n) \times 4$

Space complexity - $O(n \times n)$.

Online Stock Span - Problem - 08

arr:

100	80	60	70	60	75	85
-----	----	----	----	----	----	----

consecutive smaller
or equal

→ Before it.

op:

1	1	1	2	1	4	6
---	---	---	---	---	---	---

like if we are on 75
so check on the right 60, 70, 60 is less than 75
also there are in consecutive order so including 75 it will
be 4.

Nearest greater to left.

↓
this problem is similar to this.

0	1	2	3	4	5	6
100	80	60	70	60	75	85

If we are in the fifth idx $\uparrow$ so nearest greater to
its left will be 80 whose idx is 1

$$\text{so } 5 - 1 = \underline{\underline{4}}$$

that will be our ans.

- ① Create an array
- ② Create a stack.
- ③ Used the algo of Nearest greater to left.

Time complexity - $O(N)$

Space complexity - $O(2N)$.

Celebrity Problem - Problem - 09

Celebrity

known by everybody
knows nobody

	0	1	2	3	4
0	0	1	1	1	1
1	1	0	0	1	0
2	1	0	0	1	0
3	0	0	0	0	0
4	0	1	0	1	0

so 3 know nobody and 3 is known by everybody so its celebrity,

there can't be two celebrities and also its is possible that no body is celebrity.

3
2
1
0

pop two element.

2

3

	0	1	2	3
0	X	✓	✓	✓
1	✓	X	✓	X
2	X	X	X	X
3	✓	✓	✓	X

then check 2 is know by 3 , No

and check 3 is know by 2 , No

we will eliminate one of them so , because.

2 is known by 3 so 3 can't be a celebrity

so 2 push into the st.

2
1
0

New pop 1 an → d 2

check 1 knows 2 (yes) so it can't be a celebrity.

push 2 into st

2
0

Now pop 0 and 2

0 knows 2 (yes)

but 2 doesn't know 0 so push 2 into the stack, now $st.size == 1$ so 2 is a celebrity.

```
main(arr[]){
```

```
    Stack<int> st;
```

```
    for (i=0; i<n; i++){
```

```
        st.push(i);
```

```
    }
```

```
    while (st.size() >= 2) {
```

```
        int i = st.pop();
```

```
        int j = st.pop();
```

```
        if (arr[i][j] == 1) {
```

```
            st.push(j);
```

```
        }
```

```
    } else {
```

```
        st.push(i);
```

```
    }
```

```
    }
```

```
    int pot = st.pop();
```

```
    for (i=0; i<n; i++){
```

```
        if (i == pot) {
```

```
            if (arr[i][pot] == 0 || arr[pot][i] == 1)
```

```
                return -1;
```

```
        }
```

```
    }
```

```
    return pot;
```

Tc - $O(N)$

Sc - $O(N)$

Reverse words in a String : Problem - 90

Input: $S = \text{"The sky is blue"}$

Output: "blue is sky The"

We can follow the Brute-force initially. use a stack.
and if there is a character add it into the string and if there is any space put it into the stack and initialize $str = ""$ again.

$st = "" \rightarrow$ Because we want to add the last string as well.

```
Stack <String> st();
```

```
String str = "";
```

```
for (i=0; i<s.length; i++) {
```

```
    if (s.charAt(i) == ' ') {
```

```
        st.push(str);
```

```
        str = "";
```

```
    }
```

```
    else {
```

```
        str = s.charAt(i);
```

```
    }
```

```
}
```

```
String ans = "";
```

```
while (st.size() != 1) { // otherwise it adds a empty cell in the
```

```
    ans = ans + st.peek() + " ";
```

```
    st.pop();
```

```
}
```

```
ans += ans + st.peek();
```

```
return ans;
```

$\rightarrow$ // finally add the last element in the string.

Time complexity - $O(N)$

Space complexity - $O(N)$.

Optimal Approach

- Start traversing the String from ^{start to} the end untill we hit a space. It means we have gonna part a word.
- Check if ans variable is empty or not.
- If it's empty, it indicates that it is the last word we need to print.
- If it is not empty we add it to result with a space after it.

main fun(s) {

int left = 0, right = s.length() - 1;

String temp = ""; → for storing substring.

String ans = ""; → for storing final string.

while (left <= right) {

char ch = s.charAt(left);

if (ch != ' ') {

temp += ch;

else if (ch == ' ') {

if (!ans.equals("")) {

ans = temp + " " + ans;

else {

ans = temp;

temp = "";

left++;

}

TC - $O(N)$

SC - $O(1)$

// for adding last word

if (!temp.equals("")) {

if (!ans.equals(""))

ans = temp + " " + ans;

else ans = temp;

return ans;

}

Roman to Integer : Problem-91

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

- I can be placed for V(5) and X(10) to make 4 and 9
- X can be placed before L(50) and C(100) to make 40 and 90
- C can be placed before D(500) and M(1000) to make 400 and 900.

	↓	↗	↗	↓	↗	↗
M	C	M	X	C	I	V

- Start traverse from end initially

res = V

, 80

res = 5

then go to I

Now currently we are in I which is 1

and res is 5 mean value of (I) < res:

so simply subtract.

res = res - val of (I)

res = 5 - 1 = 4

the goto C -

on val of (C) which is 100 > res. so we have to add it to the result.

res = res + val of (C)

= 100 + 4

= 104

the goto X whose val is 10 so we need to subtract val of (X) < res.

res = res - val of (X)

= 104 - 10

= 94

then goto M so res = 1094

then goto C so res = 994

on M res = 1994

```
Map <Character, Int> ();
```

put I, V, X, L, C, D, M with its value manually.

```
int res = map.get (s.charAt(s.length()-1));
```

```
for (i = s.length()-2 ; i >= 0; i++) {
```

```
    if (map.get (s.charAt(i)) < map.get (s.charAt(i+1))) {
```

```
        result -= map.get (s.charAt(i));
```

```
    }
```

```
    else {
```

```
        result += map.get (s.charAt(i));
```

```
    }
```

```
}
```

```
return res;
```

Time complexity - $O(N)$

Space complexity - $O(N)$

String to Integer: Problem 92

atoi algorithm → • read in and ignore leading white space.

- Check for +, - if it is met in the last.
- Ignore the non digit character and return the one.
- Convert "123" → 123
"0032" → 32.
- 32 bit signed integer range $[-2^{31}, 2^{31} - 1]$. then clamp the integer so it remains in the range.

```
main fun (String s) {
```

```
    if (s == null) return 0;
```

```
    s = s.trim ();
```

```
    if (s.length() == 0) return 0;
```



```

int sign = +1;
long ans = 0;
if (s.charAt(0) == '-') sign = -1;
int MAX = Integer.MAX_VALUE;
int MIN = Integer.MIN;
int i = (s.charAt(0) == '+' || s.charAt(0) == '-') ? 1 : 0;
while (i < s.length()) {
    if (s.charAt(i) == ' ' || !Character.isDigit(s.charAt(i))) break;
    ans = ans * 10 + s.charAt(i) - '0';
    // Check
    if (sign == -1 && -1 * ans < MIN) return MIN;
    if (sign == +1 && ans > MAX) return MAX;
    i++;
}
return (int) (sign * ans);

```

Time complexity - $O(N)$.

Space complexity - $O(1)$.

Longest common prefix - Problem - 93

Str = ["Flower", "Flow", "Flight"]

o/p = "Fl"

Sort the string and compare the first and last string, so we able to find the common prefix.

Why sorting - ? The reason why we sort the input array and compare first and last strings is that the longest common prefix of all the string must be a prefix of first and last.

This is because strings are based on their alphabetical order.

like if we have -

[flower, flow, apple, flight, food]

Sort - [apple, flower, flow, flight, food]

we can clearly see the max diff between the character is between 1st and last string.

```
main fun(String[] str) {
```

```
    Arrays.sort(str);
```

```
    String s1 = str[0];
```

```
    String s2 = str[str.length - 1];
```

```
    int idx = 0;
```

```
    while (idx < s1.length() && idx < s2.length()) {
```

```
        if (s1.charAt(idx) == s2.charAt(idx)) {
```

```
            idx++;
```

```
        }
```

```
    else
```

```
        break;
```

```
}
```

```
    return s1.substring(0, idx);
```

Time complexity - $O(N \log N) + O(N)$

Space complexity - $O(1)$

Pattern searching | Rolling hash | Robin Karp - Problem - 94

$a = "abcd"$, $b = "cdabcdab"$

$O/p = 3$.

given two strings a and b , return the min number of times you should repeat string a so that b is substring of a . if impossible $\infty - 1$.

Firstly look at Robin Karp:

so we have a given text and pattern we have to check the pattern is a substring of text or not.

text $\rightarrow a \rightarrow b \rightarrow x \rightarrow a \rightarrow b \rightarrow c$

pattern $\rightarrow a \rightarrow b \rightarrow c$

Brute force -

Compare the text and pattern characters one

by one.

① first both matched the character a and a .

② Now move to pointer to next idx.

text $\rightarrow a \rightarrow b \rightarrow x \rightarrow a \rightarrow b \rightarrow c$

pattern $\rightarrow a \rightarrow b \rightarrow c$

again b matched with b , move to the next idx.

text $\rightarrow a \rightarrow b \rightarrow x \rightarrow a \rightarrow b \rightarrow c$

pattern $\rightarrow a \rightarrow b \rightarrow c$

Not matched so reset the pointer i to 0

and set the pointer j to next of previous starting $idx = 1$

text $\rightarrow a \rightarrow b \rightarrow x \rightarrow a \rightarrow b \rightarrow c$

pattern $\rightarrow a \rightarrow b \rightarrow c$

index element not match so move to $j+1$ and $i=0$

Test = a → b → x → a → b → c →

pattern = a → b → c

again Not match. move the idx. j

test = a → b → x → a → b → c

pattern = a → b → c

① j and i value match so move to next idx

② j and i _____

③ j and i _____

at end $i > \text{pattern.length}$, so it is a substring of test.

Time complexity - $O(m \times n)$

Space complexity - $O(1)$.

Optimisation -

test = a b x a b c a b c a b y

pattern = a b c a b y

We want to find the pattern in the given text. it is present or not.

① If at any index the value matches so update the pointer to the next idx. → rule

test = a b x a b c a b c a b y

pattern = a b c a b y

① matched so move to next idx.

② again the values matched. move to next idx.

test = a b x a b c a b c a b y
 pattern = a b c a b y

↑ i

Not matching the values, so now i keep moving to the left and check that there is any x or not because there is no x so i will set to 0 and j will be increase.

test = a b x a b c a b c a b y
 pattern = a b c a b y

↑ i ↑ j ↑ i ↑ j ↑ i

- ① Now values matched to move the pointers to next idx.
- ② again matched. move to next idx.
- ③ match ④ match ⑤ match.

Now j is on c and i is on y. → No matched so we checked to, mean i keep moving to the left and check there is any c or not.

test = a b x a b c a b c a b y
 pattern = a b c a b y

↑ i ↑ j ↑ i

Here we find there is a idx whose value match with j. so put the $i = i + 1$ and $j = j + 1$

test = a b x a b c a b c a b y
 pattern = a b c a b y

↑ i ↑ j ↑ i

- ① matched i and j so move to next idx
- ② matched
- ③ matched

Now $j > \text{pattern.length}$ mean its true.

Implementation -

text = a b x a b c a b c a b y
pattern = a b c a b y

We create a array for setting the longest prefix substring.

0	0	0	1	2	0
---	---	---	---	---	---

fill 0 initially.

- ① i and j not match move j to next idx
- ② i and j not match move j to next idx.
- ③ value of i and j match so put the val of a + idx + 1 previous a = 0 + 1 = 1
- ④ value of i and j match. → so get the idx → which is 1 and add 1 to it and put it 2
- ⑤ Now y not match to any one put 0.

a	b	c	a	b	y
0	0	0	1	2	0
0	1	2	3	4	5

test a b x a b c a b c a b y

Here i and j match - a match
Here _____ - b match

Now x and c are not matching, so

in the array move to the previous location and look at the val. which having val = 0
so move to idx = 0

Now a compared with x. not equal so update the
j = j + 1 because i is on the first idx.
↓
0th.

Now i and j value match so the idx increment

Now i and j _____

Now y is not matched with c

so we look at the val stored at previous idx. which is 2.

so look at idx 2.

it matches with the c so i and j will be equal / match.

so the whole string matched.

Problem: 95

Z-function | Find the idx of First occurrence in a string

```
main fun(s1, s2) {
```

```
    int n = s2.length;
```

```
    int lps[] = new int[n];
```

```
    i = 0, j = 1;
```

```
    while (j < n) {
```

```
        while (i > 0 && s1.charAt(j) != s2.charAt(i))
```

```
            i = lps[i-1];
```

```
        if (s2.charAt(i) == s2.charAt(j))
```

```
            i++;
```

```
        lps[j] = i;
```

```
        j++;
```

```
    }
```

```
    i = 0, j = 0;
```

```
    while (i < s1.length()) {
```

```
        while (j > 0 && j < n && s2.charAt(j) != s1.charAt(i))
```

```
            j = lps[j-1];
```

```
        if (j < n && s2.charAt(j) == s1.charAt(i))
```

```
            j++;
```

```
        i++;
```

```
        if (i == n) return 1; // vnicam matched..
```

```
    }
```

```
    return -1;
```

Move on to question -

```
main (s1, s2) {
```

```
StringBuilder sb ();
```

```
int rep = B.length() / A.length();
```

```
int cut = 1;
```

```
for (i = 0; i < rep + 2; i++) {
```

```
    if (sb.toString().contains(s2))  
        return cut;
```

```
    else
```

```
        res.append(A);
```

```
        cut++;
```

```
}
```

```
return -1;
```

Tc - $O(N) + O(M)$

Sc - $O(N)$] $\rightarrow$ Robin Karp

Count and Say : Problem - 98

• `countAndSay(1) = "1"`

• Break the digit into minimal numbers of substrings such that each substring contains exactly one unique digit.

"3322251"

two 3's, three 2's, one 5, one 1

= 23 + 32 + 15 + 11

= "23321511"

n = 1 "1" one ①

n = 2 "11" two 1's

n = 3 "21" two 1

n = 4 "1211"

n = 5 "111221"

Start with "1" build n-1 times.

```
main fun (int n) {
```

```
    String one = "1";
```

```
    if (n == 1) return "1";
```

```
    if (n == 2) return "11";
```

```
    for (i = 0; i < n; i++) {
```

```
        ans = build(one);
```

```
    }
```

```
    return ans;
```

```
}
```

```
// function for one build build(one) {
```

```
    int freq = 1;
```

```
    char num = charAt(0);
```

```
    ans =
```

```
    String res = "";
```

```
    for (i = 1; i < ans.length(); i++) {
```

```
        if (currNum == ans.charAt(i))
```

```
            freq++;
```

```
        else
```

```
            char freq = (char)('0' + freq);
```

```
            res = res + freq;
```

```
            res = res + num;
```

```
            currNum = ans[i];
```

```
            freq = 1;
```

```
}
```

```
    char freq = (char)('0' + freq);
```

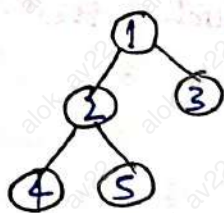
```
    res = res + freq;
```

```
    res = res + num;
```

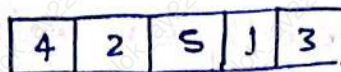
Morris Inorder Traversal - Problem 104

Inorder - Left, root, right

I/P:



Op:



```
ArrayList<int> lt;
```

```
Stack<Node> st;
```

```
Node temp = root.left;
```

```
st.push(root);
```

```
while (temp != null || !st.isEmpty()) {
```

```
    if (temp != null) {
```

```
        st.push(temp);
```

```
        temp = temp.left;
```

```
    }
```

```
    else {
```

```
        temp = st.pop();
```

```
        lt.add(temp.data);
```

```
        temp = temp.right;
```

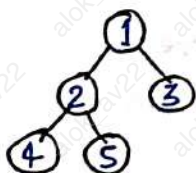
```
    }
```

```
}
```

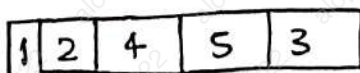
```
return lt;
```

Pre Order Morris Traversal - Problem - 105

Inorder - Root, left, right



Op:



```
ArrayList<int> lt;
```

```
Stack<Node> st;
```

```
st.push(root);
```

```
while (!st.isEmpty()) {
```

```
    Node cur = st.pop();
```

```
    lt.add(cur.data);
```

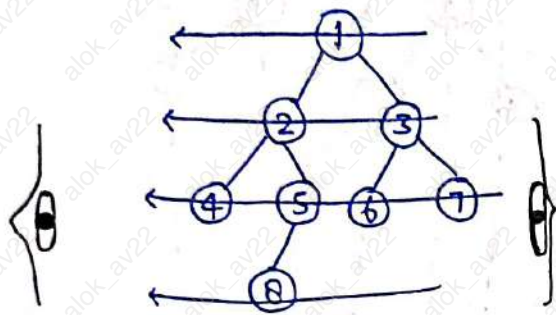


```

    if (cur.left != null)
        st.add(cur.left);
    if (cur.right != null)
        st.add(cur.right);
    return st;

```

Right/Left View of Binary Tree - Problem - 106



Time complexity - $O(N)$
 Space complexity - $O(H)$

Intuition: We have to do a recursive level order Traversal.

Root Right Left $\rightarrow$ for Right View

Root Left Right $\rightarrow$ for Left view

```

public class Solution {
    public List<Integer> rightSideView (TreeNode root) {
        List<Integer> res = new ArrayList<>();
        helperRightSideView (root, res, 0);
        return res;
    }

    public void helperRightSideView (TreeNode cur, List<Integer> res, int dep) {
        if (cur == null) return;

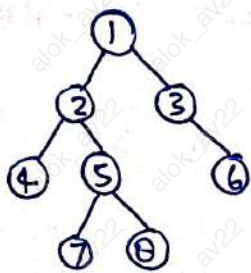
        if (depth == res.size())
            res.add (cur.val);

        helperRightView (cur.right, res, dep+1);
        helperRightView (cur.left, res, dep+1);
    }
}

```

} swap for left view.

Bottom View of Binary Tree



Bottom View -

4	7	5	8	6
---	---	---	---	---

BottomView(root) {

 ArrayList<int> res;

 if (root == null) return res;

 Map<int, int> treemap;

 Queue<Node> q;

 root.hd = 0;

 q.add(root);

 while (!q.isEmpty()) {

 Node temp = q.poll();

 int hd = temp.hd;

 map.put(hd, temp.data); // for Top view check

 if (temp.left != null)

 temp.left.hd = hd - 1;

 q.add(temp.left);

 if (temp.right != null)

 temp.right.hd = hd + 1;

 q.add(temp.right);

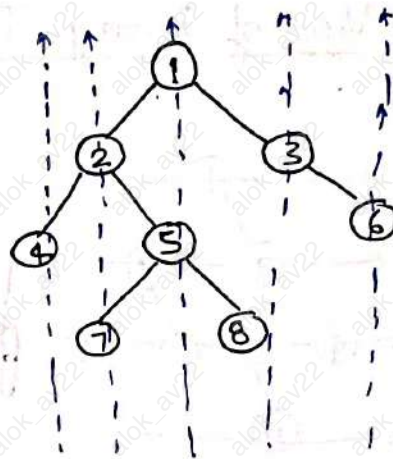
 }

 for (Map.Entry<int, int> entry : map.entrySet()) {

 res.add(entry.getValue());

 }

 return res;

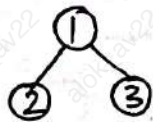


if (map.containsKey(hd) == false)

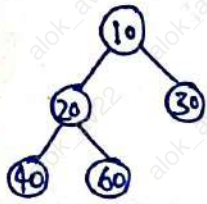
Time complexity - $O(H)$

Space " - $O(N)$

Root to Leaf Paths



O/p: 1 2 # 1 3



I/p: O/p: 10 20 40 # 10 20 60 # 10 30

ArrayList<ArrayList<int>> ans;

ArrayList<int> lt;

main(Node root)

help(root);

return ans;

help(Node root) {

if (root == null) return;

lt.add(root.data);

if (root.left == null & root.right == null)

ans.add(new ArrayList<>(lt));

help(root.left);

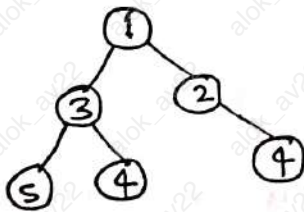
help(root.right);

}

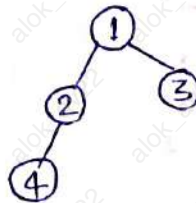
Time complexity - $O(N)$

Space complexity - $O(N)$

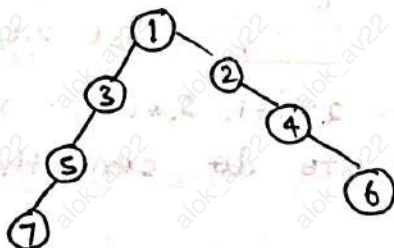
Maximum width of Binary Tree.



Max width = 4



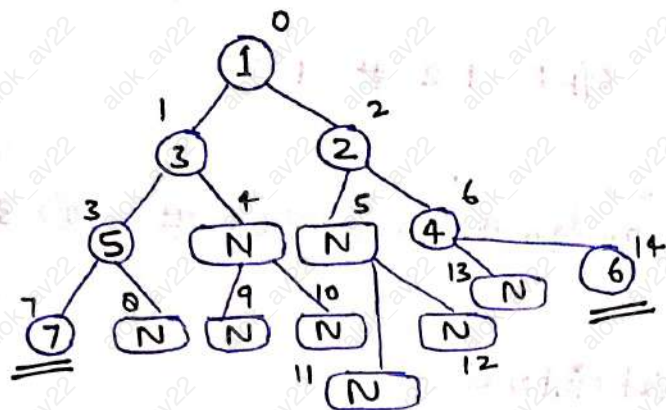
max width = 2



Max width = 3

width for a level is define as the maximum number of nodes between the left most and right most node of level.

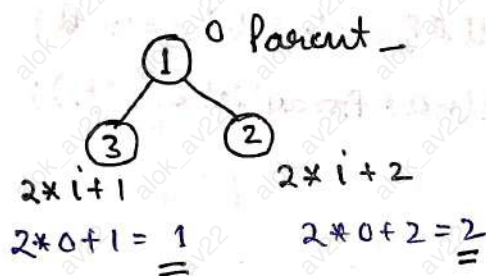
Intuition:



Max width = 8

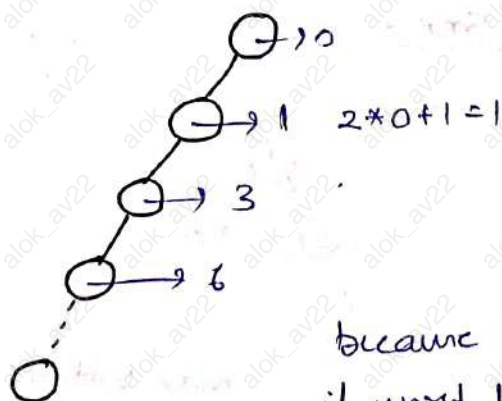
See the left most and right most node at every level. For achieve this we need unique id for every Node. Once we know the left most and right most nodes width can be decide as $(\text{rightMost} - \text{leftMost} + 1)$.

A parent given $2 \times i + 1$ to left child and $2 \times i + 2$ to the right child.



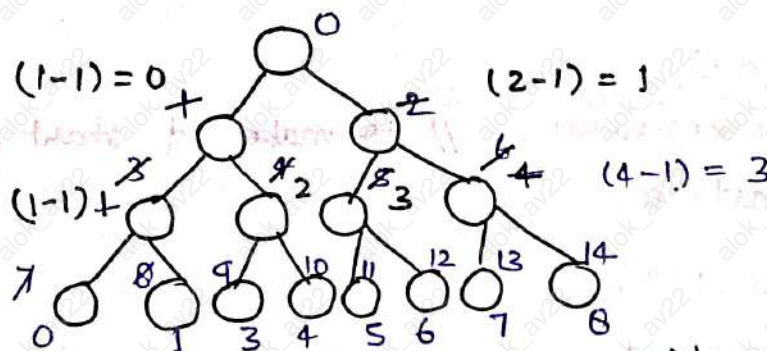
Similarly for other.

But this fails on tc.

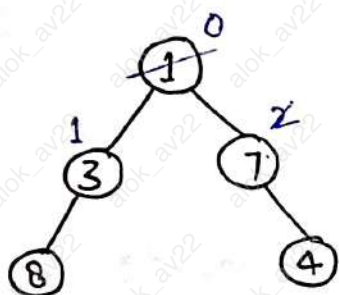


because increasing in twice if worst length 105 so it will overflow. so we can't use exactly $2 \times i + 1$, $2 \times i + 2$. instead we need to do something else.

How to prevent overflow



Now we can see that we handle the overflow condition.



(4, 4)
(8, 0)
(7, 2)
(3, 1)
(1, 0)

width = 2

$$(3, 1) \rightarrow (1-1) = 0+1 = 1$$

$$(7, 2) \rightarrow (2-1) = 1$$

$$(last - first + 1)$$

$$(2-1) + 1 = 2$$

```
class TreeNode {
    int data;
    TreeNode left, right;
    TreeNode (int data) {
        this.data = data;
        left = null;
        right = null;
    }
}
```

```
class Pair {
    TreeNode node;
    int num;
    Pair (TreeNode node, int num) {
        this.node = node;
        this.num = num;
    }
}
```

```
main (root) {
    if (root == null) return 0;
    int ans = 0;
    Queue < Pair > q = new LinkedList ();
    q.add (new Pair (root, 0));
}
```

```
while (!q.isEmpty()) {
```

```
    int size = q.size();
```

```
    int mmin = q.peek().num; // to make id start from 0.
```

```
    int first = 0, last = 0;
```

```
    for (i = 0; i < size; i++) {
```

```
        int cur_id = q.peek().num - mmin;
```

```
        TreeNode node = q.peek().node;
```

```
        q.poll();
```

```
        if (i == 0) first = cur_id;
```

```
        if (i == size - 1) last = cur_id;
```

```
        if (node.left != null)
```

```
            q.add(new Pair(node.left, cur_id * 2 + 1));
```

```
        if (node.right != null)
```

```
            q.add(new Pair(node.right, cur_id * 2 + 2));
```

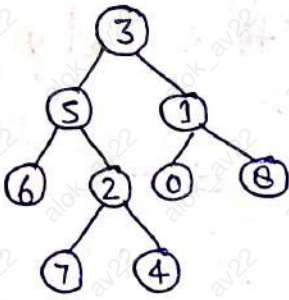
```
    }  
    ans = (ans, last - first + 1);  
    max }
```

```
return ans;
```

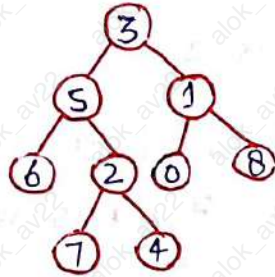
```
}
```


lowest common Ancestor of a Binary Tree

↓
the lowest parent of two Nodes.



$p=5, q=1$, the LCA will be 3



$p=5, q=4$
lowest CA = 5

Intuition - we can observe that we can find the LCA of 2 given nodes from

- ① left subtree or right subtree
- ② If not in both, the root will be LCA.

1. If root is null or if root is x or if root is y then return root.

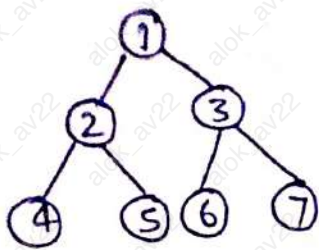
2. Make a recursion call for both. ① Left
② Right

★ Make a recursive call → left subtree

↓
if null, mean LCA not found on left subtree which means we found LCA on right subtree. so return Right.

★ Make a recursive call → right subtree

↓
if null, mean LCA not found on right subtree which means we found LCA on left so return left.



$$x = 4, y = 5$$

root is null if not equal to x and y
so recursion call.

call left subtree it find 4 so this call will return 4 to its parent.

At present the parent is 2.

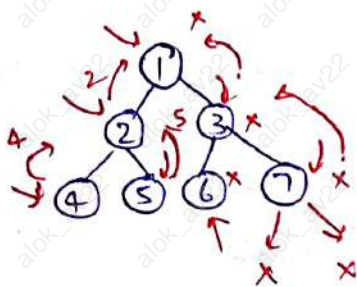
★ Call the right subtree, while calling it will find 5.
and this call will return 5 to its parent.

Now left recursive call returns val. not null
↓

Right recursive call " " "



thus 2 will return itself to root i.e. to 1.



$$lca(4, 5) = 2$$

Bruteforce - take the root to node path to x and y.

↓ ↓ ↓
(1, 2, 4)

↑ ↑ ↑
(1, 2, 5)

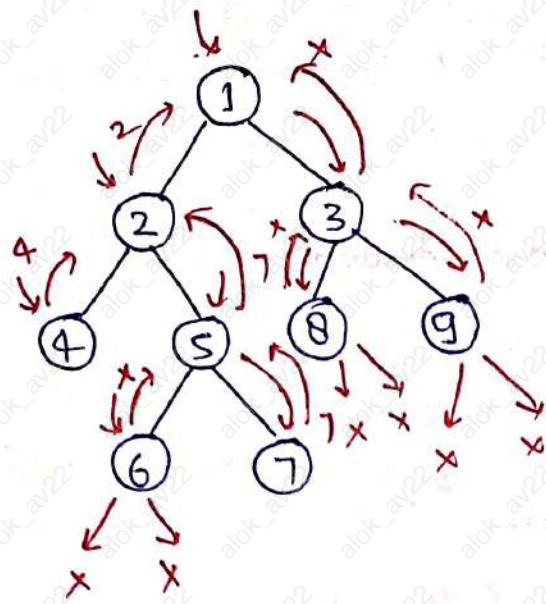
at zeroth idx match

at 1st idx match

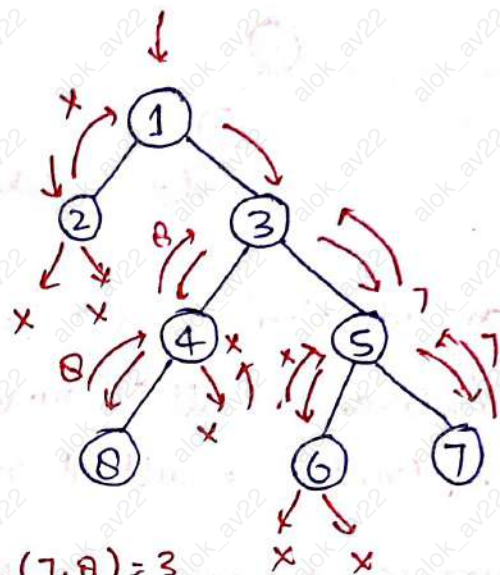
not matched on 2nd so

last match was 2 so it is LCA.

$O(N)$ → Space & Time



$$lca(4, 7) = 2$$



$$lca(7, 8) = 3$$

both the node we got so we are confirm that this the the lca we are looking for

main `LCA()` {

if (root == null || root == p || root == q) return root;

TreeNode left = LCA (root.left, p, q);

TreeNode right = LCA (root.right, p, q);

// result

if (left == null) return right;

if (right == null) return left;

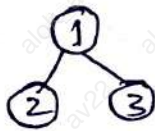
else // if left and right both != null we found the result.
return root;

Check if two trees are identical -

1



node 1



node 2

both are identical.

2



node 1



node 2

not identical.

boolean identical (node 1, node 2) {

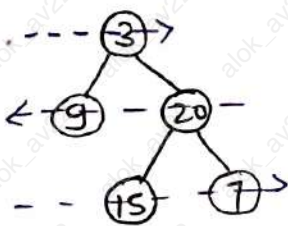
if (node 1 == null && node 2 == null) return true;

if (node 1 == null || node 2 == null) return false;

return ((node 1.data == node 2.data) && identical (node 1.left, node 2.left) && identical (node 1.right, node 2.right));

}

Binary Tree Zig Zag level order Traversal.



o/p: [[3], [20, 9], [15, 7]]

main levelordertraversal (root) {

List < List <int>> lt;

if (root == null)

return lt;

Stack < Node> st1;

Stack < Node> st2;

st1.push(root);

TreeNode cur;


```
boolean lftor = true;  
while (!st1.isEmpty() || !st2.isEmpty()) {
```

```
    List<int> level();
```

```
    if (lftor) {
```

```
        while (!st1.isEmpty()) {
```

```
            curr = st1.pop();
```

```
            level ← ltor.add(curr.val);
```

```
            if (curr.left != null)
```

```
                st2.push(curr.left);
```

```
            if (curr.right != null)
```

```
                st2.push(curr.right);
```

```
        }
```

```
    }
```

```
    else {
```

```
        while (!st2.isEmpty()) {
```

```
            curr = st2.pop();
```

```
            level ← ltor.add(curr.val);
```

```
            if (curr.right != null) st1.push(curr.right);
```

```
            if (curr.left != null) st1.push(curr.left);
```

```
        }
```

```
    }
```

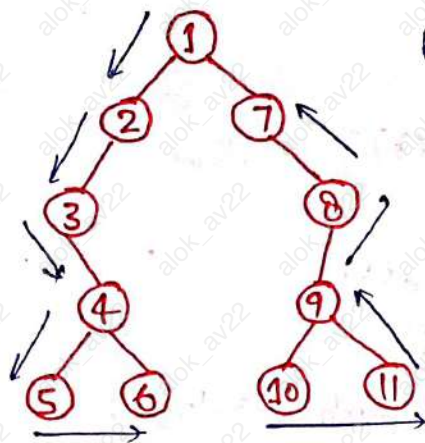
```
    ans.add(ltor);
```

```
    ltor = !lftor;
```

```
    return ans;
```

```
}
```

Boundary Traversal of Binary Tree.



Boundary traversal:

1	2	3	4	5	6	10	11	9	8	7
---	---	---	---	---	---	----	----	---	---	---

1. Part 1: Left Boundary of tree.
2. Part 2: All the leaf nodes travelled in the left to right
3. Part 3: Right boundary of tree (exclude leaf) reverse.

```

printBoundary(node) {
    ArrayList<int> ans ();
    if (isLeafNode == false) ans.add (node.data);
    addLeftBoundary (node, ans);
    addLeaves (node, ans);
    addRightBoundary (node, ans);
    return ans;
}
    
```

```

Boolean isLeaf (node) {
    return (root.left == null) && (root.right == null);
}
    
```

```

addLeftBoundary (node, ans) {
    Node cur = root.left;
    while (cur != null) {
        if (isLeaf (cur) == false) ans.add (cur.data);
        if (isLeaf (cur) == false) cur = cur.right;
        else cur = cur.left;
    }
}
    
```



```
if (isLeaf(cur) == false) one.add(cur.data);
```

```
if (cur.left != null) cur = cur.left;
```

```
else
```

```
cur = cur.right;
```

```
}
```

```
addRightBoundary(root, one) {
```

```
Node cur = root.right;
```

```
ArrayList<int> tmp = new ArrayList<>();
```

```
while (cur != null) {
```

```
if (isLeaf(cur) == false) tmp.add(cur.data);
```

```
if (cur.right != null) cur = cur.right;
```

```
else
```

```
cur = cur.left;
```

```
}
```

```
int i;
```

```
for (i = tmp.size() - 1; i >= 0; i--) {
```

```
one.add(tmp.get(i));
```

```
}
```

```
}
```

```
addLeaves(root, one) {
```

```
if (isLeaf(root))
```

```
one.add(root.data);
```

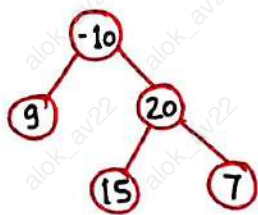
```
return;
```

```
if (root.left != null) addLeaves(root.left, one);
```

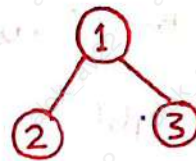
```
if (root.right != null) addLeaves(root.right, one);
```

```
}
```

★ Binary Tree maximum Path sum



most optimal maximum path
sum = $15 + 20 + 7 = \underline{42}$

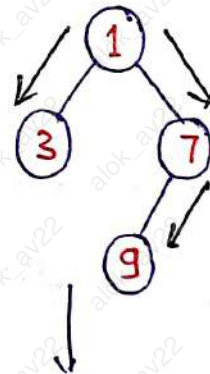
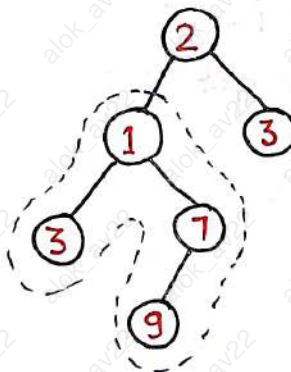
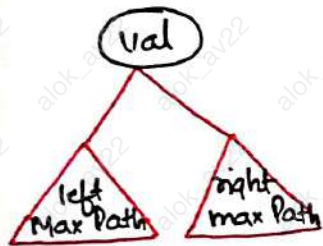


$$2 + 1 + 3 = 6$$

A brute force approach would be to generate all paths and compare them. Generating all paths will be a time costly activity therefore we need to look for some thing else.

At a given node with a val, if we find the max left sum path in the left subtree and right subtree,

$$\text{maxPathSum} = \text{val} + \text{leftSumPath} + \text{rightSumPath}$$



- Initialize a maximize to store final one.

- Do a Simple tree traversal.
At each node, find recursive its left Max Path and right Max sum path.

- cal. the val + left max Path + right Max sum and update maxi accordingly.

- Return the max Path when node is next the curving point as val + max(left Max Path, rightMaxPath).

$$\text{left path} = 3$$

$$\text{right path} = 1 + 7 + 9 = 16$$

$$\text{max Path Sum} = \text{val} + \text{left path} + \text{right path}$$

$$= 3 + 1 + 16$$

$$= 20$$

maxPathSum (root) {

int maxVal[] = new int[1];

maxVal[0] = Integer.MIN-VALUE;

maxPathDown (root, maxVal);

return maxVal[0];

}

maxPathDown (node, maxVal) {

if (node == null) return 0;

int left = Math.max(0, maxPathDown (node.left, maxVal));

int right = Math.max(0, maxPathDown (node.right, maxVal));

[maxVal[0] = Math.max (maxVal[0], left + right + node.val);]

return Math.max (left, right) + node.val;

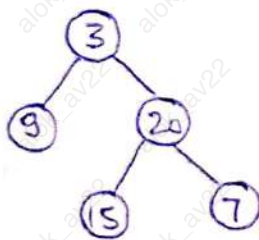
}

↳ return to its parent.

backtrack
logic.

Construct a Binary Tree from inorder and pre order.

Problem - 120



Input: [3, 9, 20, 15, 7]

output: [9, 3, 15, 20, 7]

output: [3, 9, 20, null, null, 15, 7]

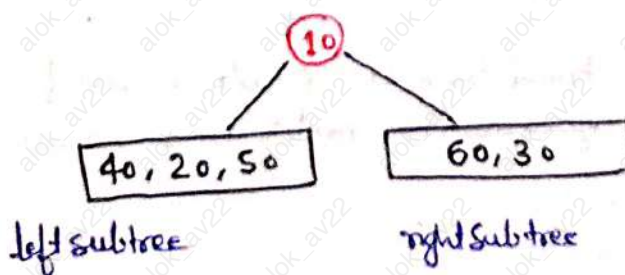
Input: inorder → [40, 20, 50, 10, 60, 30]

preorder → [10, 20, 40, 50, 30, 60]

↑
root

left, root, right

root, left, right.



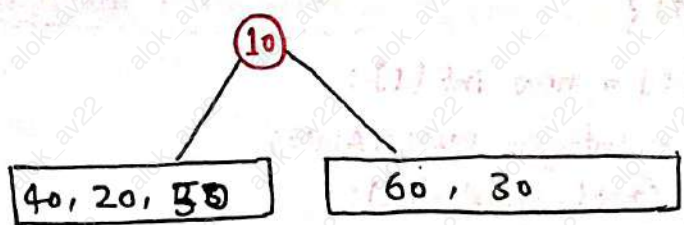
left subtree

right subtree

order can be anything
we only consider the left
and right part

Inorder →

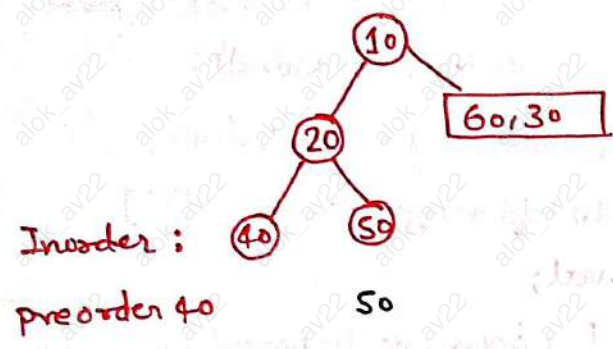
Preorder →



→ Inorder

20, 40, 50

30, 60



Inorder :

preorder 40

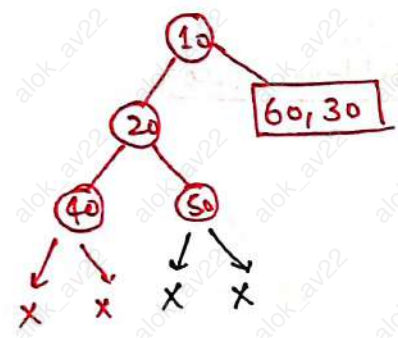
50

Inorder → 40, 20, 50

preorder - 20, 40, 50

L, Root, R

Root, L, R

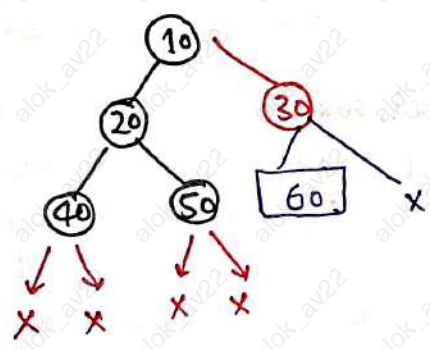


Inorder - 40

preorder - 40

Inorder: 50

preorder: 50

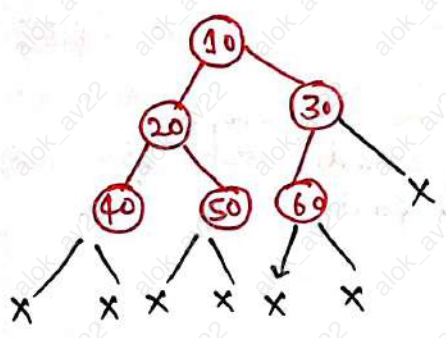


Inorder - 60, 30

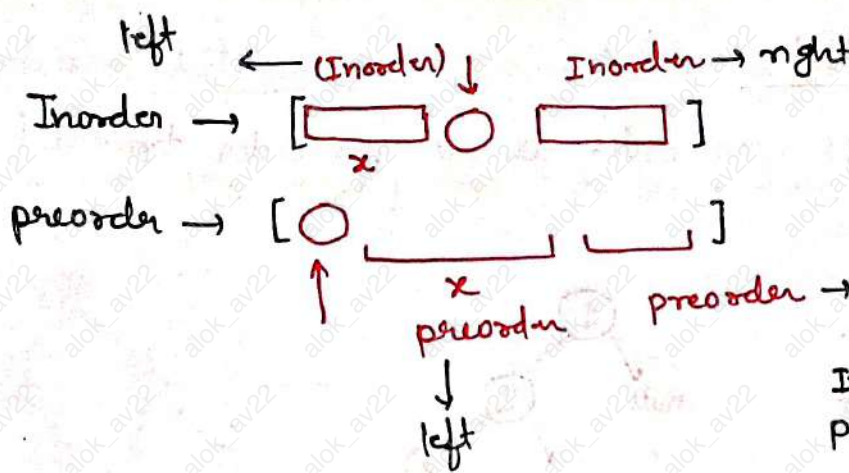
preorder - 30, 60

Inorder - 60

preorder - 60



breakdown the problem and generate the subtree.



Inorder: 9, 3, 15, 20, 7
 Preorder: 3, 9, 20, 15, 7

9	→	0
3	→	1
15	→	2
20	→	3
7	→	4

```
main(preorder[], inorder[]) {
    HashMap<int, int> map();
    for (i = 0; i < inorder.length; i++) {
        map.put(inorder[i], i);
    }
}
```

```
TreeNode root = buildTree(preorder, 0, preorder.length-1, inorder, 0,
                           inorder.length-1, map);
```

```
return root;
```

```
}
```

```
buildTree(preorder, int preStart, int preEnd, inorder[], int inStart,
           int inEnd, map) {
```

```
    if (preStart > preEnd || inStart > inEnd) return null;
```

```
    TreeNode root = new TreeNode(preorder[preStart]);
```

```
    int inRoot = map.get(root.val);
```

```
    int numLeft = inRoot - inStart;
```

```
    root.left = buildTree(preorder, preStart+1, preStart+numLeft, inorder,
                           inStart, inRoot-1, map);
```

```
    root.right = buildTree(preorder, preStart+numLeft+1, preEnd, inorder,
                           inRoot+1, inEnd, map);
```

```
    return root;
```

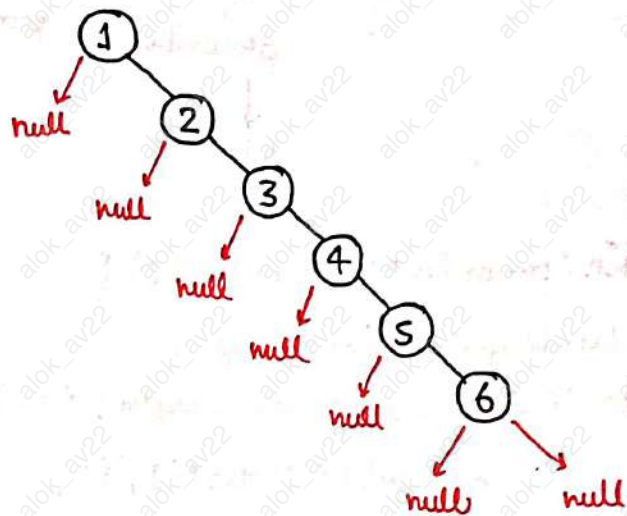
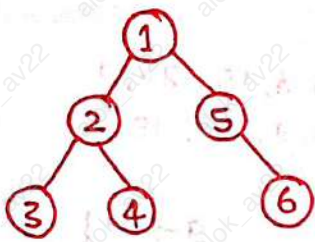
```
}
```

Tc - $O(N)$

Sc - $O(N)$

Flatten Binary Tree to linkedlist.

The linkedlist should be in the same order as a pre-order traversal of Binary tree.



class Solution {

static Node pre = null;

static void flatten(Node root) {

if (root == null) return;

flatten (root.right);

flatten (root.left);

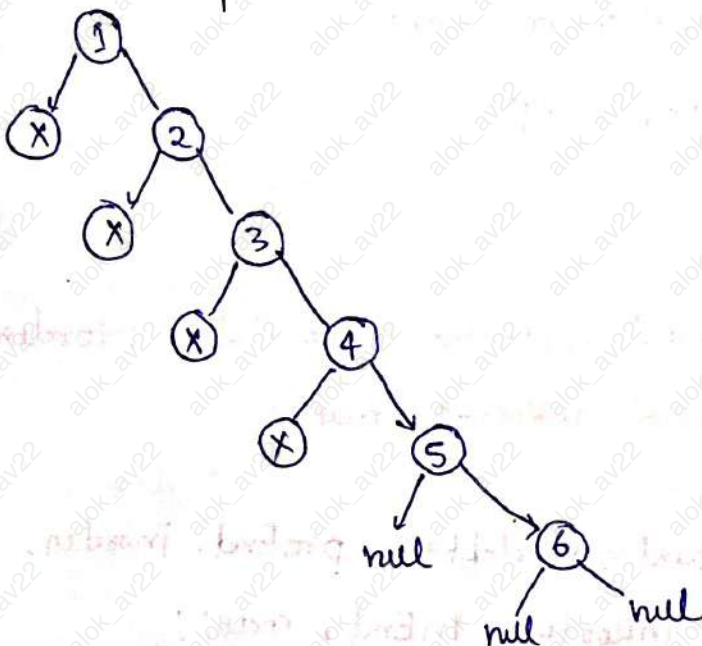
root.right = pre

root.left = null;

pre = root;

}

}



pre = null

1, left, null

1, right, null

2, left, 3

2, right, 4

3, left, 2

6, left, null

6, right, null

5, right, 6

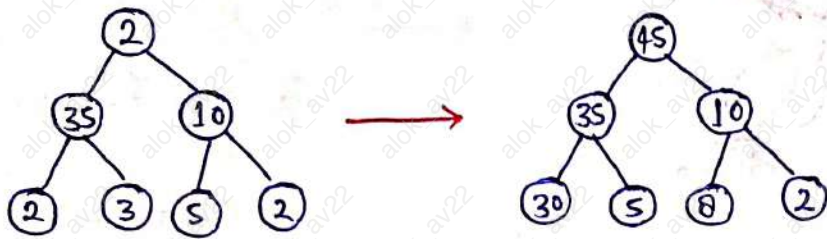
1, right, 5

Root, left, right

↑
Similarly

Check for children sum property in a Binary Tree.

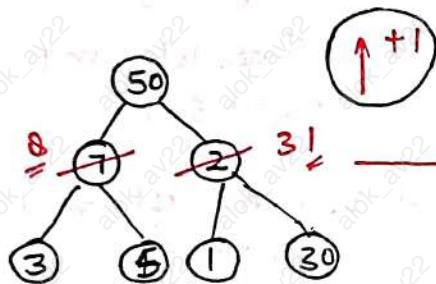
convert the binary Tree into sum tree -



$$\text{node} = \text{left} + \text{right}$$

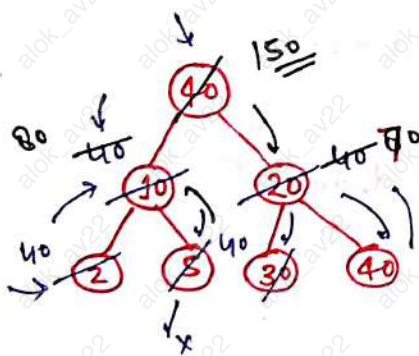
Note - A val for null node can be accessed 0

2. Not allowed to change the structure.



Sheet 11

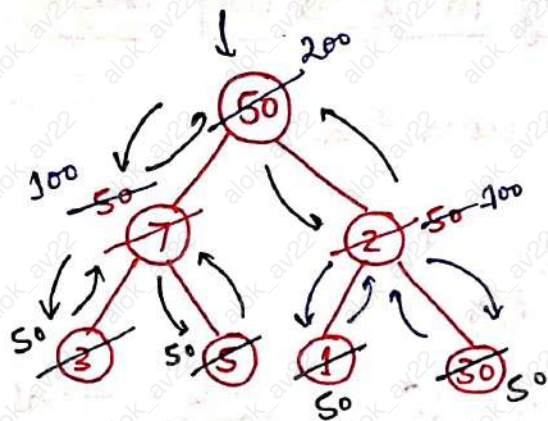
so we can see over here the sum of left and right $8+31=41$, and the root is 50, so we can't minimize the root val., that's the problem this is not a simple problem.



$$10 + 20 = 30 < 40$$

$$2 + 5 = 7 < 40$$

$$30 + 40 = 70 > 40 \text{ (make root = left + right)}$$



```
main() {
```

```
    if (root == null) return;
```

```
    int child = 0;
```

```
    if (root.left != null) {
```

```
        child += root.left.data;
```

```
    } if (root.right != null) {
```

```
        child += root.right.data;
```

```
    } if (root.data <= child)
```

```
        root.data = child;
```

```
    else {
```

```
        if (root.left != null)
```

```
            root.left.data = root.data;
```

```
        else if (root.right != null)
```

```
            root.right.data = root.data;
```

```
    }
```

```
    selffunrecursion (root.left);
```

```
    selffunrecursion (root.right);
```

```
    int tot = 0;
```

```
    if (root.left != null)
```

```
        total += root.left.data;
```

```
    if (root.right != null)
```

```
        total += root.right.data;
```

```
    if (root.left != null || root.right != null)
```

```
        root.data = total;
```

```
}
```

$7 + 2 = 9 < 50$ [make left and right 50]

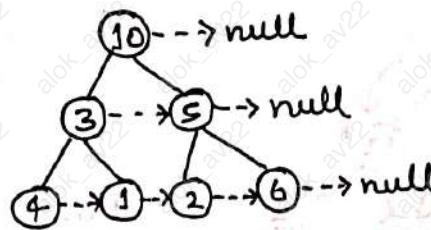
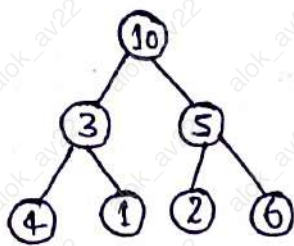
$3 + 5 = 8 < 50$ [make left and right 50]

$1 + 30 = 31 < 50$ [make left and right 50]

Tc - $O(N)$

Sc - $O(H)$

Connect nodes at the same level.



connect(root) {

Queue <Node> q ();

q.add(root);

Node temp = null;

while (!q.isEmpty()) {

int n = q.size();

for (i = 0; i < n; i++) {

Node pre = temp;

temp = q.poll();

if (i > 0) {

pre.nextRight = temp;

}

if (temp.left != null)

q.add(temp.left);

if (temp.right != null)

q.add(temp.right);

}

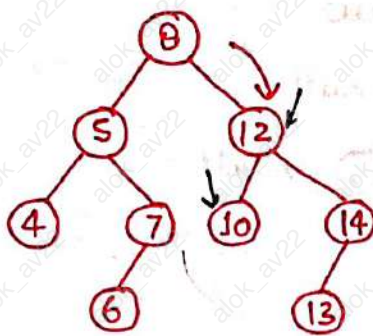
temp.nextRight = null

}

Time complexity - $O(N)$

Search In a Binary Tree -

BST $\rightarrow (\log_2 N)$



node = 10

$L < N < R$

It is a BST so we can see that 8 is less than 10 so 8 is less than 10 so 10 could be lie in right.

12 is greater than 10 so go to 10

$10 == val$

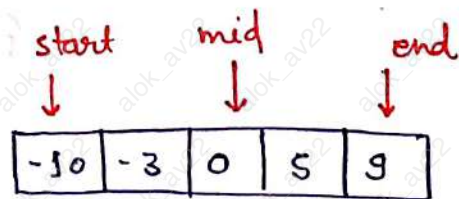
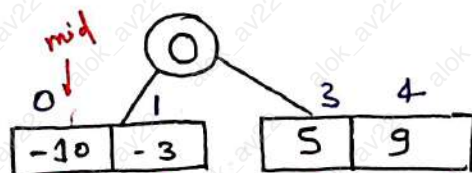
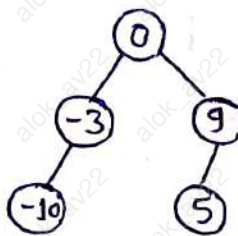
if reach null return null:

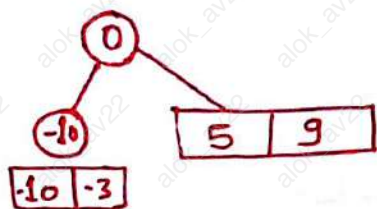
```
while (root != null && root.val != val) {  
    root = val < root.val ? root.left : root.right;  
}  
return root;
```

Convert sorted Array to Binary Search Tree -

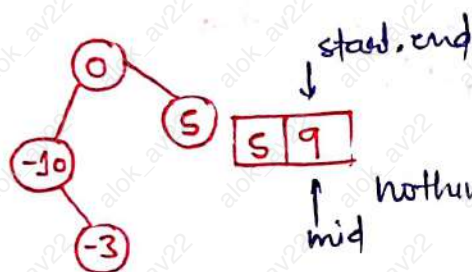
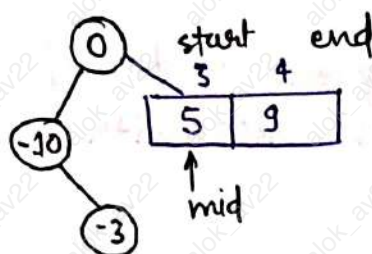
arr = [-10, -3, 0, 5, 9]

clearly show the root will be the mid element.



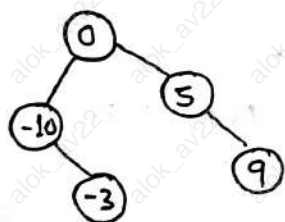


there is nothing on the left of -10 make a recursive call on right side.



nothing on the left of 5.

So recursive call at right side.



```
main(arr) {
```

```
    return (help(arr, 0, arr.length - 1));
```

```
}
```

```
help(arr, start, end) {
```

```
    if (start > end) return null;
```

```
    Node node = new Node(arr[mid]);
```

```
    node.left = help(arr, start, mid - 1);
```

```
    node.right = help(arr, mid + 1, end);
```

```
    return node;
```

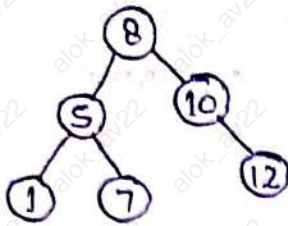
```
}
```

```
int mid = (start + end) / 2;
```

BST from pre-order Traversal.

preorder -

8	5	1	7	10	12
---	---	---	---	----	----



Better solution - we know that preorder traversal is given and if we know the Inorder so we can construct the BST with help of both the traversal.

preorder -

8	5	1	7	10	12
---	---	---	---	----	----

Inorder -

1	5	7	8	10	12
---	---	---	---	----	----

} So we can create the BST by this.

Tc - $O(N \log N) + O(N)$

Sc - $O(N)$

Optimal solution -

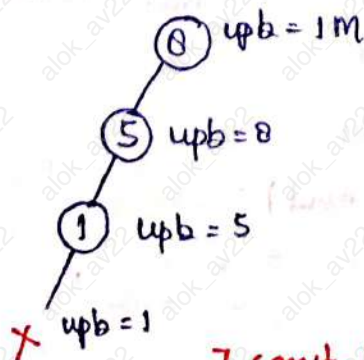
take the upperbound. initially as 1M.

firstly check 8 is under the upperbound or not (yes it is).

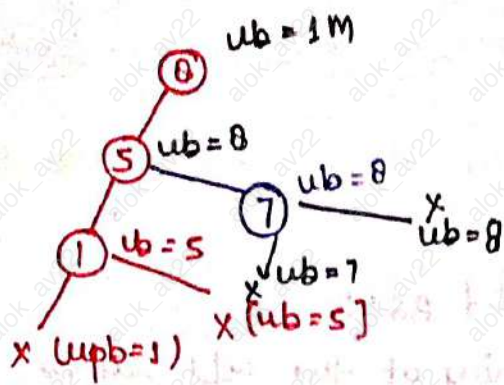
⑧ upb = 1M

we don't know how many element that will lie at the left side of 8.

We know one thing for sure. everything at the left will always smaller than 8.

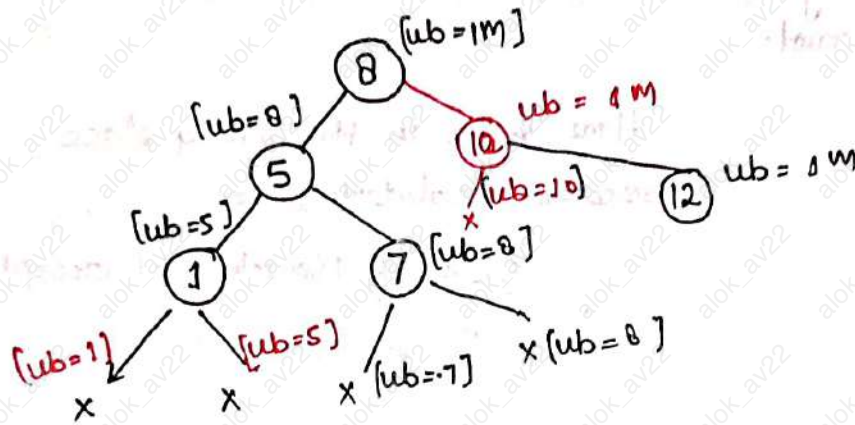


7 can't be on left. so it assign null



↓

8	5	1	7	10	12
---	---	---	---	----	----



If we are calling left, then we have to pass that val. as a Bound.
 $\{(left, node.val)\}$

If we are calling right, then we have to the val. which is prev. Bound.

$\{(right, bound)\}$

Tc - we are traversing a single node 3 times.

Tc - $O(3N) \approx O(N)$

Sc - $O(1)$.

```
main(int arr){
```

```
    return help(arr, INT_MAX, new int[] {0});
```

```
    }
```

```
    help(arr, bound, int[] i){
```

```
        if(i[0] == arr.length || arr[i[0]] > bound) return
```

```
        TreeNode root = new TreeNode(A[i[0]+1]);
```

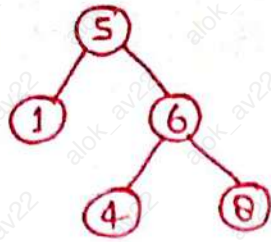
```
        root.left = help(arr, root.val, i);
```

```
        root.right = help(arr, bound, i);
```

```
        return root;
```

```
    }
```

Check BT is BST or not.



The valid BST:

- * everything at the left has to be smaller than that node
- * and everything at the right has to be greater.

By this we the intuition come in our mind we will pass a range and ask every node hey, you will lie at this range or not.

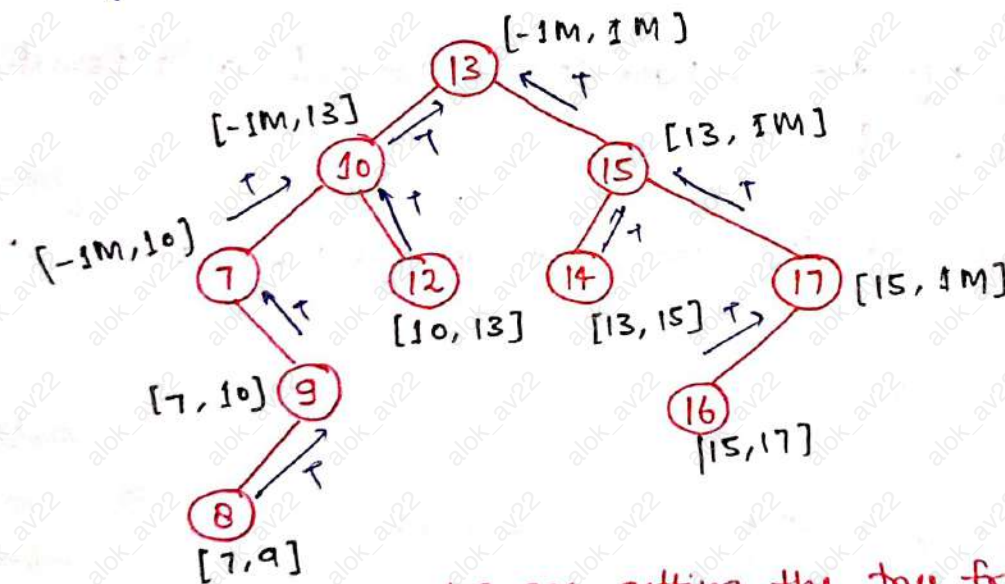
Range - $[L, H]$

Here 4 is in the wrong place because validation is.

$$5 < 4 < 6 \text{ (which not accepted).}$$

Range $[5, 6]$

initial range $[-1M, 1M]$



we are getting the tree from both

the subtrees.

So its a valid BST.

Tc - $O(N)$

Sc - $O(1)$

main(root) { return help(root, LONG_MIN, LONG_MAX); }

help(root, long minval, long maxval) {

if (root == null) return true;

if (root.val >= maxval || root.val <= minval) return false;

return help(root.left, minval, root.val) && help(root.right,

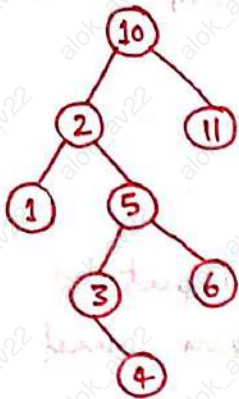
root.val, maxval);

}

Predecessor and Successor:

A key is given to us and we have to search in the tree for predecessor and successor

if not found set it to null.



key = 0, output = [6, 10]

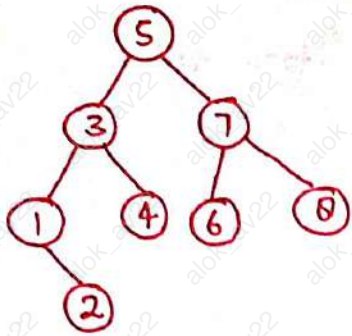
It is a bst so we can decide that we have to go to the left subtree/right subtree.

```
main(root) {  
    predecessor(root, key);  
    successor(root, key);  
}
```

```
predecessor(root, key) {  
    while (root != null) {  
        if (root.data >= key) {  
            root = root.left;  
        }  
        else {  
            pre = root.data;  
            root = root.right;  
        }  
    }  
}
```

```
successor(root, key) {  
    while (root != null) {  
        if (root.data <= key) {  
            // successor = root.data;  
            root = root.left;  
            root = root.right;  
        }  
        else {  
            successor = root.data;  
            root = root.left;  
        }  
    }  
}
```

Kth smallest element in a BST - / largest element



$K=3$, we have to find the 3rd smallest solution.

Naive solution - take an Arraylist and do the traversal put all the element of tree into the list and sort them and in the last get the K^{th} element.

Tc - $O(N) + O(N \log N)$

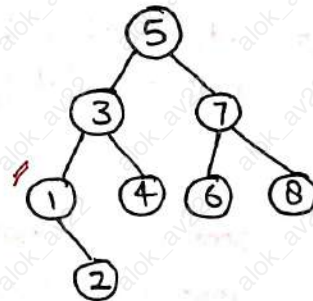
Sc - $O(N)$

Efficient solution

Inorder - Left, node, Right.

1, 2, 3, 4, 5, 6, 7, 8

inorder traversal



If we use the inorder traversal so it given all the element in a sorted order.

by this we remove $O(N \log N)$ for sorting.
Now our task is to remove $O(N)$ extra space.

take a cnt variable and make sure if we are visiting the node then we have to increase the cnt. by 1.

if (cnt == k)
 ans = node

Tc - $O(N)$
Sc - $O(N)$ } Recursion

if we have to find k^{th} largest:

one traversal - $O(N)$

k^{th} largest = $(N - k^{\text{th}})$ smallest.



```
kthSmallest(root, int k) {  
    return help(root, new int[] {k});  
}
```

```
help(root, int k[]) {  
    if (root == null) return null;  
    Node left = help(root.left, k);  
    if (left != null)  
        return left;  
    k[0]--;  
    if (k[0] == 0) return root;  
    return help(root.right, k);  
}
```

Logic - 1

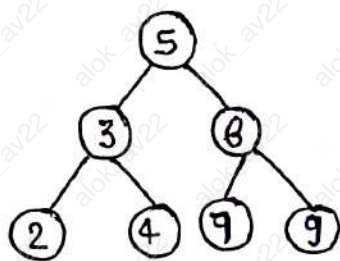
```
private int cnt = 0;  
private int res = 0;
```

```
kthSmallest(root, k) {  
    traverse(root, k);  
    return result;  
}
```

```
traverse(root, k) {  
    if (root == null) return null;  
    traverse(root.left, k);  
    cnt++;  
    if (cnt == k)  
    {  
        res = root.data;  
        return;  
    }  
    traverse(root.right, k);  
}
```

Logic - 2

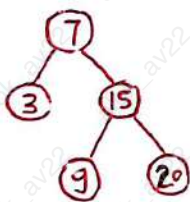
Pair with given Sum in BST.



we can solve this problem using Hashset.

```
static Set<int> st;  
main (root, k) {  
    st = new HashSet<>();  
    return help(root, k);  
}  
help (root, k) {  
    if (root == null) return false;  
    if (st.contains(root.val + k)) return true;  
    st.add(root.val);  
    return help(root.left, k) || help(root.right, k);  
}
```

Binary Search Iterator.



so we have to design a class of functions so we can operate this operation.

BSTIterator(7) → root
next → inorder's guy.

next → 3

next → 7

hasnext → yes.

next → 9

hasnext → yes

next → 15

hasnext → yes

next → 20

hasnext → no

if we take an array store all the integers so we can perform the operations easily.



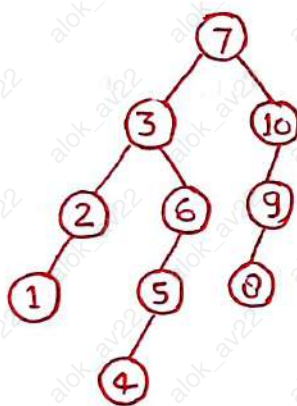
next → 3

hasnext → yes

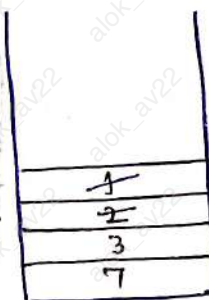
it will be easy.

But the TC - $O(1)$ and SC - $O(N)$, because we are storing all the values in the array.

Inorder - L, N, R



Class → BSTIterator(T)



Stack.

we will go in extreme left, so we will do in the construct we will go in extreme left and put everything into stack

Now if some one calls 1's next, as it's null, so check the stack's first element.

next - 1

when again next called, does 2 have a right?

next - 2

check 3 has a right?

Yes it has

next → 3

Go to right,

Now put everything into stack.

Similarly 5 and 4 added because left is called for 6.

4
5
6
7

Now 4 left will be null.

Now if some say. hasNext?

empty, then return true;

Just check if stack is not

next → 4

4 doesn't have a right

next → 5

5 " " "

next → 6

6 " " "

next → 7

yes 7 have a right, put all the element

8
9
10

Similar things will happen -

SC - $O(H)$

TC - $O(1)$

class BSTIterator {

private Stack < TreeNode > st = new Stack < > ();

public BSTIterator (root) {

pushAll (root);
}

private void pushAll (root) {

for (; node != null; st.push (node), node = node.left;)
{


```

public boolean hasNext () {
    return ! st.isEmpty ();
}

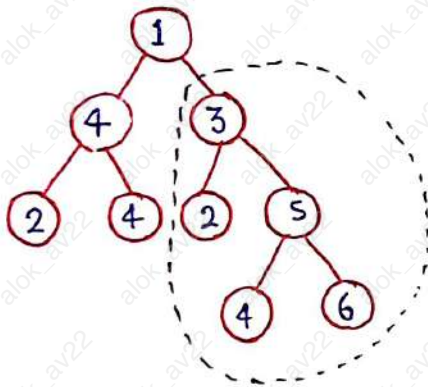
```

```

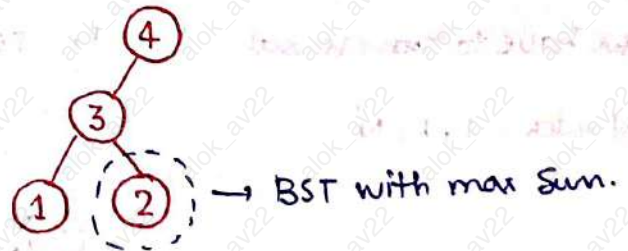
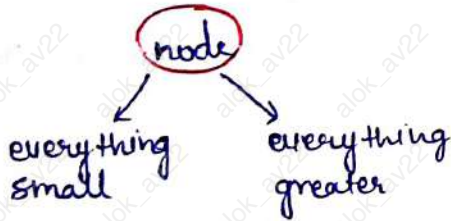
public int next () {
    TreeNode node = st.pop ();
    pushAll (node.right);
    return node.val;
}

```

Maximum Sum in BST in a Binary Tree -



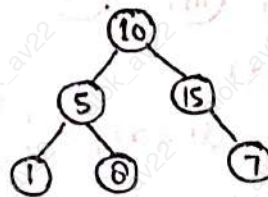
this is a BST with a max sum.



Brute force - I will validate the BST.

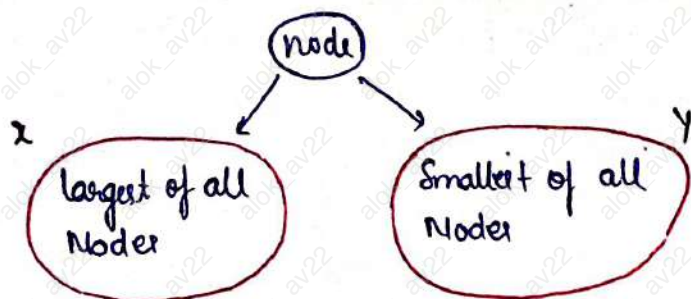
So take a min max and validate by going on every node

10 → validate BST.
5 ← yes



validate BST → $O(N) \times O(N) = O(N^2)$

↓
going to every node



$$\text{Size} = 1 + x + y$$

$$\text{largest} < \text{node} < \text{smallest}$$

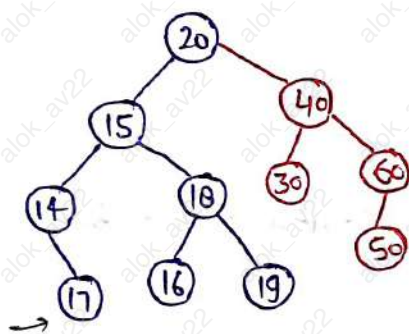
We will go to bottom and we know the bottom guy will be a BST → for sure. and it's connected to a node.

Idea - Build the things from bottom.



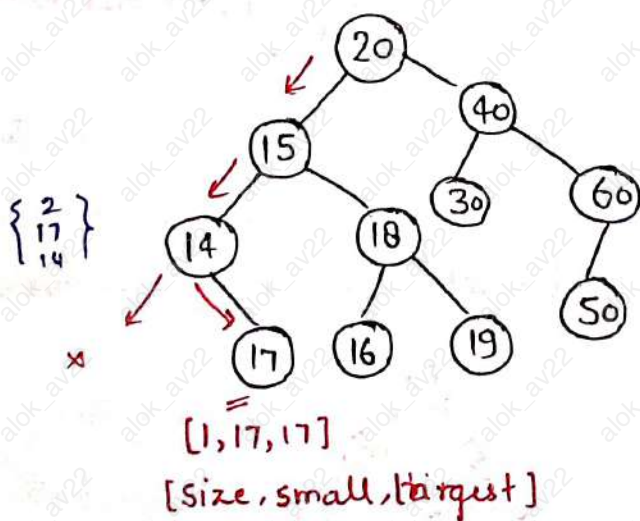
if satisfied the condition the BST

store the largest, smallest and size.



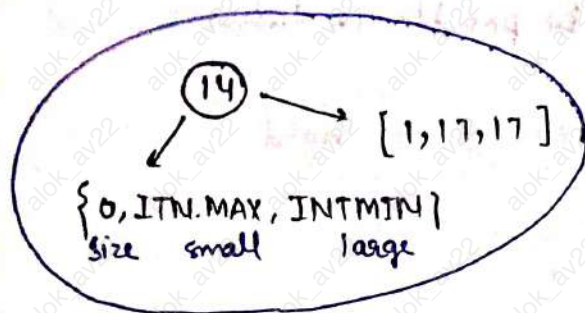
We have to transversal L, R, N so we take all element, L and R

postorder - L, R, N



= → BST

if there is no node on left and right so pass the dummy node of size → 0, for largest take INT-MIN and for smallest take INT-MAX.

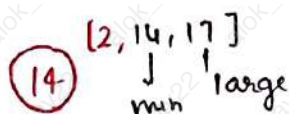


→ so when will be this will be a valid BST.

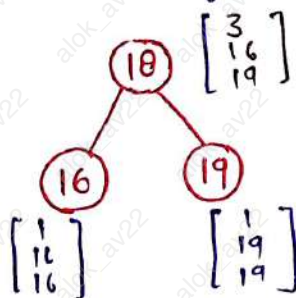
for 14 to be a valid BST.

$$\text{INTMIN} < 14 < 17$$

for 14 → size will be $1+x+y \Rightarrow 1+0+1 \Rightarrow 2$



Now for 15 the left is completed.



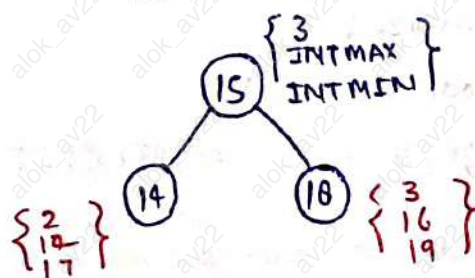
so if come to 18 so check.
node, which is 18. is greater than largest of left subtree
and smaller than the smallest of right subtree

$16 < 18 < 19 \rightarrow$ yes, so 18, will contain BST.

Size for 18 $\rightarrow 1+x+y = 1+1+1 = 3$

Now come to 15.

At the moment we come to 15., look at largest of left 17
and minimal of right 16



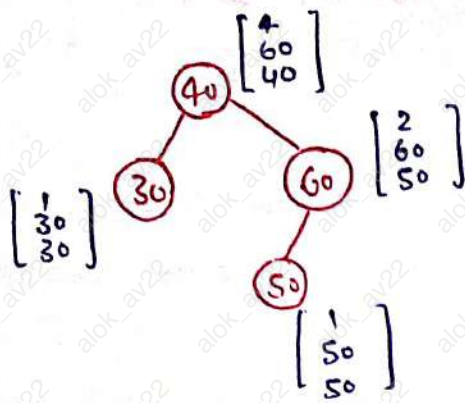
$17 < 15$ (x not following)

right is OK but left not

Not in that condition pass the INTMIN and MAX

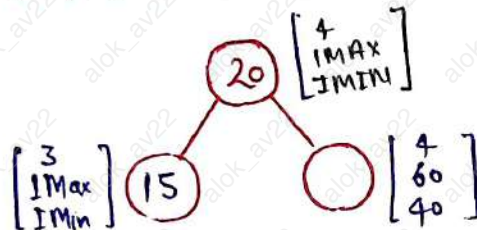
we can MIN and MAX comparison won't be possible in future

Now every thing in left of 20 completed go to right.



$$30 < 40 < 50 \text{ (true)}$$

At moment we came 20.



$$\frac{INTMAX < 20 < 40}{x \quad \checkmark}$$

But largest BST

if BST. size = 1 + x + y otherwise = max(left, right)

Tc - $O(N)$

SC $\rightarrow O(1)$

```
class NodeValue {
```

```
    public int maxNode, minNode, maxSize;
```

```
    NodeValue (int maxNode, minNode, maxSize) {
```

```
        this.maxNode = maxNode;
```

```
        this.minNode = minNode;
```

```
        this.maxSize = maxSize;
```

```
    }
```

```
}
```


class Solution {

```
main(root) {
```

```
    if (root == null) {
```

```
        return new NodeValue (INT-MAX, INT-MIN, 0);  
    }
```

```
    Node left = main_(root.left);
```

```
    Node right = main_(root.right);
```

```
    if (left.maxNode < root.val && root.val < right.minNode) {
```

```
        return new NodeValue (Math.min (root.val, left.minNode), Math.max (root.val,  
                                                                           right.maxNode),  
                               (left.maxSize + right.maxSize + 1));
```

```
    return new NodeValue (INTMIN, INTMAX, max (left.maxSize, right.maxSize))  
    }
```

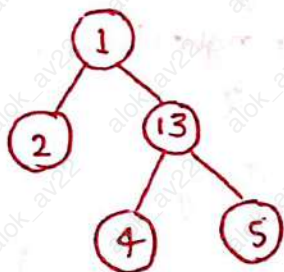
```
main () {
```

```
    return main - (root).maxSize;
```

```
}
```

```
}
```

Serialize and Deserialize Binary Tree -



root

Serialize

→ root → String

Deserialize

reverse
logic

1, 2, 13, #, #, 4, 5, #, #, #, #

Make sure there have a null on leaf node

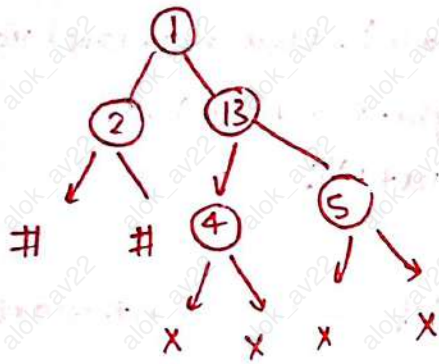
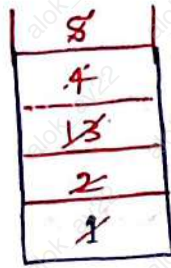
Now we have serialize -

the string we get: 1, 2, 13, #, #, 4, 5, #, #, #, #

↑ ↑

Now for deserialize -

take queue ds.



```
public String serialize(TreeNode) {
```

```
    if (root == null) return "";
```

```
    Queue<TreeNode> q = new LinkedList<>();
```

```
    StringBuilder res = new StringBuilder();
```

```
    q.add(root);
```

```
    while(!q.isEmpty()) {
```

```
        TreeNode node = q.poll();
```

```
        if (node == null) {
```

```
            res.append("n ");
```

```
            continue;
```

```
        }
```

```
        res.append(node.val + " ");
```

```
        q.add(node.left);
```

```
        q.add(node.right);
```

```
    }
```

```
    return res.toString();
```

```
}
```

```
public TreeNode deserialize(String data) {
```

```
    if (data == "") return null;
```

```
    Queue<TreeNode> q = new LinkedList<>();
```

```
    String[] values = data.split(" ");
```



```

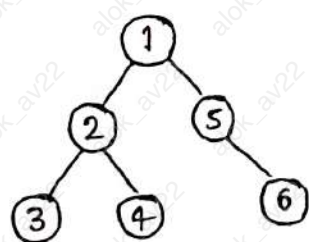
TreeNode root = new TreeNode (Integer.parseInt (values[0]));
q.add (root);

for (int i = 1; i < values.length; i++) {
    TreeNode parent = q.poll ();
    if (! values [i]. equals ("n")) {
        TreeNode left = new TreeNode (Integer.parseInt (values[i]));
        parent.left = left;
        q.add (left);
    }
    if (! values [++i]. equals ("n")) {
        TreeNode right = new TreeNode (Integer.parseInt (values[i]));
        parent.right = right;
        q.add (right);
    }
}

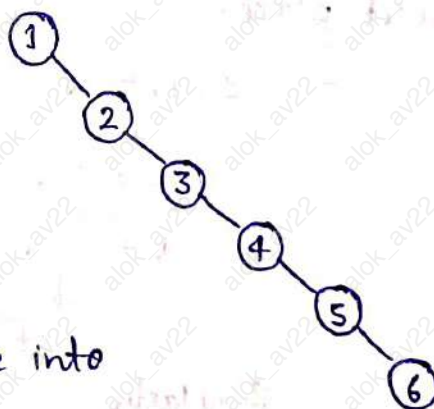
return root;
}

```

Flatten Binary Tree to Linkedlis -



o/p →



We have to change the Binary Tree into linkedlist.

The thing we have to observe -

so 6 is at the bottom.

If we see the output at the left

The conditions -

- linkedlist should use the same `TreeNode` class where the right child pointer points to the next node in the list and the left child pointer is always null.

class Solution {

 TreeNode pre = null;

 flatten(root) {

 if (root == null) return;

 flatten(root.right);

 flatten(root.left);

 root.right = pre;

 root.left = null;

 pre = root;

 }

}

Find Median from Data Stream.

The Number are coming in the array stream and at any moment it can ask us for the midiam.

Median → We can find the median if arr is sorted mean if new Number is added so it has to be add at the exact and correct place.

3, 1, 5, 4, ...

a → 3, median = 3

a = 3, 4

↓

sort → 1, 3

median = 2

Similarly.

a = 1, 3, 4, 5 median = 3.5

so for putting all the Number in the stream in the exact place to make sorted -

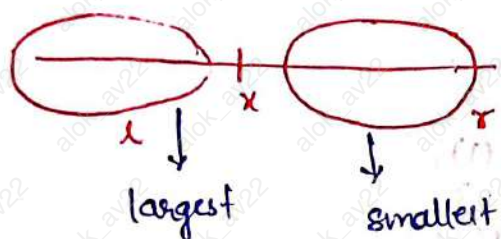
Tc - $O(N)$ and there are N Number so it will take - $O(N^2)$

Optimize -

Observation is we don't have to sort this we just need to return the median, and we know median is the middle number.

smaller < median < greater

if x is a median so the Number on right side will be equal to left side.



so we can find the median with the help of left part largest number and right part smallest number.

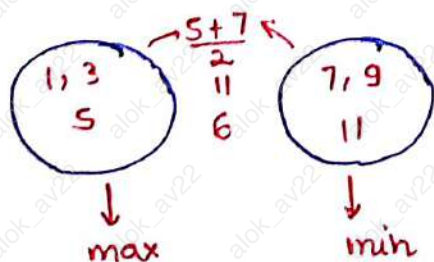
we know we can use maxHeap for getting the max Element

Similarly
for min Heap.

if number is odd to $\frac{n}{2} + 1$ element will on left heap and other added to minHeap.

if even so works like as before.

1, 3, 5, 7, 9, 11 → even



if odd so median will largest of maxHeap.

```
class Main {
```

```
    PriorityQueue<int> maxHeap = new PriorityQueue<> (Collections.reverseOrder());
```

```
    PriorityQueue<int> minHeap = new PriorityQueue<> ();
```

```
    insertNum (int num) {
```

```
        if (maxHeap.isEmpty() || maxHeap.peek() >= num) {  
            maxHeap.add(num);
```

```
        } else {  
            minHeap.add(num);  
        }
```

```
        if (maxHeap.size() > minHeap.size() + 1) {
```

```
            minHeap.add(maxHeap.poll());
```

```
        } else if (maxHeap.size() < minHeap.size()) {
```

```
            maxHeap.add(minHeap.poll());
```

```
        }
```

```
    double findMedian () {
```

```
        if (maxHeap.size() == minHeap.size()) {
```

```
            return maxHeap.peek() / 2.0 + minHeap.peek() / 2.0;
```

```
        }  
        return maxHeap.peek();
```

```
    }
```